Stack & Queue, and Sliding Window — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

169. Implement Stack using Arrays

Understanding the Problem NEW

A Stack is a 'last in, first out' (LIFO) structure — think of a stack of plates: you can only add a plate to the top, and you can only remove the plate that's currently on top. Implement this using a plain array as the backing storage.

Brute Force Approach NEW

There isn't a 'brute' alternative here — this is about learning the building block itself. The natural question is just how to simulate LIFO behaviour using an array, which normally supports access to any index.

Why We Optimize NEW

You only need **one extra variable**: a `top` index tracking the current top of the stack. Push means placing the new value at `arr[top+1]` and incrementing top; pop means reading `arr[top]` and decrementing top. Both operations are O(1) — no shifting of other elements is needed, since you only ever touch the position right at the current top, unlike inserting/deleting in the middle of an array.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement Stack data structure using array.

Visual Diagram

```
Stack: LIFO (Last In First Out)

       Operations:
       push(5): [5]
       push(3): [5,3]
       push(8): [5,3,8]
       pop(): [5,3] → returns 8
       peek(): returns 3
       isEmpty(): false

       Array Implementation:
       Index: 0 1 2 3 4
       Value: [5, 3, 8, _, _]
```

```
       Top = 2

       push(10):
       Index: 0 1 2 3 4
       Value: [5, 3, 8, 10, _]
       Top = 3
```

Solution Approach

Concept: Use array with top pointer.

Algorithm

```
Algorithm STACK_ARRAY:
           arr = new int[capacity]
           top = -1

           push(x):
               If top == capacity-1: Overflow
               arr[++top] = x

           pop():
               If top == -1: Underflow
               Return arr[top--]

           peek():
               If top == -1: Empty
               Return arr[top]
```

Java Implementation

```
class StackArray {
           private int[] arr;
           private int top;
           private int capacity;

           StackArray(int capacity) {
               this.capacity = capacity;
               arr = new int[capacity];
               top = -1;
           }

           public void push(int x) {
               if (top == capacity - 1) {
                   throw new RuntimeException("Stack Overflow");
               }
               arr[++top] = x;
           }

           public int pop() {
```

```
            if (top == -1) {
                throw new RuntimeException("Stack Underflow");
            }
            return arr[top--];
        }

        public int peek() {
            if (top == -1) {
                throw new RuntimeException("Stack is Empty");
            }
            return arr[top];
        }

        public boolean isEmpty() {
            return top == -1;
        }

        public int size() {
            return top + 1;
        }
    }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(n)

170. Implement Queue using Arrays

Understanding the Problem NEW

A Queue is a 'first in, first out' (FIFO) structure — like a line at a checkout counter: whoever joined first gets served first. Implement this using an array.

Brute Force Approach NEW

A naive array-based queue might always insert at the end and remove from the front by shifting every remaining element left by one each time — O(n) per removal, since arrays aren't naturally suited to removing from the front cheaply.

Why We Optimize NEW

Instead of physically shifting elements after every removal, use a **circular array**: track a `front` and `rear` index that both wrap around back to the start of the array once they reach the end (using modulo arithmetic). This reuses array slots that have already been dequeued, without ever needing to shift remaining elements — both enqueue and dequeue become O(1), instead of O(n) for the shifting approach.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement Queue data structure using array.

Visual Diagram

```
Queue: FIFO (First In First Out)

       Operations:
       enqueue(5): [5]
       enqueue(3): [5,3]
       enqueue(8): [5,3,8]
       dequeue(): [3,8] → returns 5
       front(): returns 3
       isEmpty(): false

       Circular Array Implementation:
       front = 0, rear = -1, size = 0
```

```
       enqueue: rear = (rear+1) % capacity
       dequeue: front = (front+1) % capacity
```

Solution Approach

Concept: Circular array to utilize space.

Algorithm

```
Algorithm QUEUE_ARRAY:
           arr = new int[capacity]
           front = 0, rear = -1, size = 0

            enqueue(x):
                If size == capacity: Overflow
                rear = (rear + 1) % capacity
                arr[rear] = x
                size++

            dequeue():
                If size == 0: Underflow
                val = arr[front]
                front = (front + 1) % capacity
                size--
                Return val
```

Java Implementation

```
class QueueArray {
           private int[] arr;
           private int front;
           private int rear;
           private int size;
           private int capacity;

            QueueArray(int capacity) {
                this.capacity = capacity;
                arr = new int[capacity];
                front = 0;
                rear = -1;
                size = 0;
            }
            public void enqueue(int x) {

              if (size == capacity) {
                  throw new RuntimeException("Queue Overflow");
              }
              rear = (rear + 1) % capacity;
              arr[rear] = x;
              size++;
```

```
        }

        public int dequeue() {
            if (size == 0) {
                throw new RuntimeException("Queue Underflow");
            }
            int val = arr[front];
            front = (front + 1) % capacity;
            size--;
            return val;
        }

        public int front() {
            if (size == 0) {
                throw new RuntimeException("Queue is Empty");
            }
            return arr[front];
        }

        public boolean isEmpty() {
            return size == 0;
        }
    }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(n)

171. Implement Stack using Queue

Understanding the Problem NEW

Implement a Stack (LIFO) using only Queues (FIFO) as your building blocks — an exercise in adapting one structure's behaviour using a fundamentally different one.

Brute Force Approach NEW

A naive approach might use just one queue and, on every push, dequeue and re-enqueue everything else after adding the new item, but doing this awkwardly on every single operation without a clear plan often leads to O(n) work per push *and* confusion about correctness.

Why We Optimize NEW

The clean approach: on every push, enqueue the new element into the queue, then **rotate the whole queue** by dequeuing and immediately re-enqueuing every element that came before it — this has the effect of moving the newest element to the very front of the queue. This makes push cost O(n) (since you rotate the whole queue) but pop trivially O(1) (just dequeue the front, which is always the most recently pushed element) — a deliberate trade-off, since the two structures have opposite natural strengths (queue front-access is cheap, but LIFO order requires 'faking' it via rotation).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement Stack using two queues.

Visual Diagram

```
Stack using Queues (Push Efficient):
       Push: Add to queue2, then move all from queue1
       Pop: Remove from queue1

       Push 5:
       q1: [5], q2: []

       Push 3:
```

```
       q2: [3], move q1: [5] → q2: [3,5]
       Swap: q1: [3,5], q2: []

       Push 8:
       q2: [8], move q1: [3,5] → q2: [8,3,5]
       Swap: q1: [8,3,5], q2: []

       Pop(): 8 from q1
       q1: [3,5]

       Stack using Queues (Pop Efficient):
       Push: Add to q2, if q1 not empty move all
       Pop: Remove from q1
```

Solution Approach

Concept: Use two queues with swapping.

Algorithm (Push Efficient)

Algorithm STACK_USING_QUEUE:

q1, q2 = Queue()

push(x):

q2.enqueue(x)

While q1 not empty:

q2.enqueue(q1.dequeue())

Swap(q1, q2)

pop():

Return q1.dequeue()

peek():

Return q1.front()

Java Implementation

```
class StackUsingQueue {
           private Queue<Integer> q1;
           private Queue<Integer> q2;

           StackUsingQueue() {
               q1 = new LinkedList<>();
               q2 = new LinkedList<>();
```

```
        }

        // Push Efficient

        public void push(int x) {
            q2.add(x);
            while (!q1.isEmpty()) {
                q2.add(q1.remove());
            }
            Queue<Integer> temp = q1;
            q1 = q2;
            q2 = temp;
        }

        public int pop() {
            if (q1.isEmpty()) {
                throw new RuntimeException("Stack is Empty");
            }
            return q1.remove();
        }

        public int peek() {
            if (q1.isEmpty()) {
                throw new RuntimeException("Stack is Empty");
            }
            return q1.peek();
        }
    }
```

Time & Space Complexity

Time Complexity: Push O(n), Pop O(1) | Space Complexity: O(n)

172. Implement Queue using Stack

Understanding the Problem NEW

The reverse of the previous problem: implement a Queue (FIFO) using only Stacks (LIFO) as building blocks.

Brute Force Approach NEW

Similarly, you could reverse the entire first stack into a second stack on every single enqueue or dequeue operation to fix the ordering — correct but doing this on every operation is wasteful if operations are interleaved unpredictably.

Why We Optimize NEW

Use **two stacks** — an 'input' stack and an 'output' stack. Enqueue always just pushes onto the input stack (O(1)). Dequeue checks the output stack first: if it's empty, pour the *entire* input stack into it (which naturally reverses the order, turning LIFO into FIFO order), then pop from the output stack; if the output stack already has elements, just pop from it directly. The key insight is that each individual element only ever gets moved from input to output **once** over its lifetime, so even though a single dequeue call might occasionally cost O(n) (when the transfer happens), the **amortized** cost across many operations averages out to O(1) per operation — a valuable concept (amortized analysis) worth understanding here for the first time in this book.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement Queue using two stacks.

Visual Diagram

```
Queue using Stacks (Enqueue Efficient):
       Enqueue: Push to stack1
       Dequeue: If stack2 empty, move all from stack1 to stack2

       Enqueue 5: s1: [5], s2: []
       Enqueue 3: s1: [5,3], s2: []
       Enqueue 8: s1: [5,3,8], s2: []
```

```
       Dequeue():
         Move s1 to s2: s1: [], s2: [8,3,5]
         Pop from s2: 5 → s2: [8,3]
         Return 5

       Queue using Stacks (Dequeue Efficient):
       Enqueue: Push to stack2, move to stack1 if needed
       Dequeue: Pop from stack1
```

Solution Approach

Concept: Two stacks with transfer on demand.

Algorithm (Enqueue Efficient)

Algorithm QUEUE_USING_STACK:

s1, s2 = Stack()

enqueue(x):

s1.push(x)

dequeue():

If s2.isEmpty():

While s1 not empty:

s2.push(s1.pop())

Return s2.pop()

Java Implementation

```
class QueueUsingStack {
           private Stack<Integer> s1;
           private Stack<Integer> s2;

           QueueUsingStack() {
               s1 = new Stack<>();
               s2 = new Stack<>();
           }

           // Enqueue Efficient
           public void enqueue(int x) {
               s1.push(x);
           }

           public int dequeue() {
               if (s2.isEmpty()) {
```

```
                while (!s1.isEmpty()) {
                    s2.push(s1.pop());

                }
            }
            if (s2.isEmpty()) {
                throw new RuntimeException("Queue is Empty");
            }
            return s2.pop();
        }

        public int front() {
            if (s2.isEmpty()) {
                while (!s1.isEmpty()) {
                    s2.push(s1.pop());
                }
            }
            if (s2.isEmpty()) {
                throw new RuntimeException("Queue is Empty");
            }
            return s2.peek();
        }
    }
```

Time & Space Complexity

Time Complexity: Enqueue O(1), Dequeue amortized O(1) | Space Complexity: O(n)

173. Implement Stack using LinkedList

Understanding the Problem NEW

Implement a Stack using a LinkedList instead of an array as the backing storage.

Brute Force Approach NEW

Using an array-backed stack (problem 169) works fine, but arrays have a fixed capacity that needs resizing (typically doubling and copying, an O(n) operation) once they fill up.

Why We Optimize NEW

A LinkedList naturally grows one node at a time with no pre-allocated capacity limit — pushing means adding a new node at the **head** (O(1), no shifting or resizing needed), and popping means removing the head node (O(1)). This avoids the occasional O(n) resize cost that array-backed implementations can incur, at the modest expense of a little extra memory per element for the 'next' pointer.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement Stack using LinkedList.

Visual Diagram

```
LinkedList Stack:
       Head → [5] → [3] → [8] → null
               ↑
              Top

       push(10):
       Head → [10] → [5] → [3] → [8] → null

       pop():
       Head → [5] → [3] → [8] → null
       Returns 10

       Node Structure:
       class Node {
           int data;
           Node next;
```

```
            Node(int data) { this.data = data; }
        }
```

Solution Approach

Concept: Use LinkedList with head as top.

Algorithm

```
Algorithm STACK_LL:
            head = null

            push(x):
                newNode = Node(x)
                newNode.next = head
                head = newNode

            pop():
                If head == null: Underflow
                val = head.data
                head = head.next
                Return val
```

Java Implementation

```
class StackLinkedList {
            private static class Node {
                int data;
                Node next;
                Node(int data) {
                    this.data = data;
                }
            }

            private Node head;

            public void push(int x) {
                Node newNode = new Node(x);
                newNode.next = head;

               head = newNode;
            }

            public int pop() {
                if (head == null) {
                    throw new RuntimeException("Stack Underflow");
                }
                int val = head.data;
                head = head.next;
                return val;
```

```
        }

        public int peek() {
            if (head == null) {
                throw new RuntimeException("Stack is Empty");
            }
            return head.data;
        }

        public boolean isEmpty() {
            return head == null;
        }
    }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(n)

174. Implement Queue using LinkedList

Understanding the Problem NEW

Implement a Queue using a LinkedList instead of an array.

Brute Force Approach NEW

An array-backed queue (problem 170) using the circular-array trick works well, but still has a fixed maximum capacity that needs resizing occasionally.

Why We Optimize NEW

With a LinkedList, keep two references — `head` (front of queue, where you dequeue) and `tail` (rear, where you enqueue) — so both enqueue (add after tail) and dequeue (remove from head) are O(1), with no capacity limit and no need for the circular-index wraparound trick that an array implementation requires. The trade-off, as with problem 173, is a bit more memory overhead per element for the pointer fields.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement Queue using LinkedList.

Visual Diagram

```
LinkedList Queue:
       Head → [5] → [3] → [8] → null ← Tail

       enqueue(10):
       Head → [5] → [3] → [8] → [10] → null ← Tail

       dequeue():
       Head → [3] → [8] → [10] → null ← Tail
       Returns 5

       Node Structure:
       class Node {
           int data;
           Node next;
```

```
            Node(int data) { this.data = data; }
        }
```

Solution Approach

Concept: Use head for front, tail for rear.

Algorithm

```
Algorithm QUEUE_LL:
            head = null, tail = null

             enqueue(x):
                 newNode = Node(x)
                 If tail == null:
                     head = tail = newNode
                 Else:
                     tail.next = newNode
                     tail = newNode

             dequeue():
                 If head == null: Underflow
                 val = head.data
                 head = head.next
                 If head == null: tail = null
                 Return val
```

Java Implementation

```
class QueueLinkedList {
            private static class Node {
                int data;
                Node next;
                Node(int data) {
                    this.data = data;
                }
            }

             private Node head;
             private Node tail;
             public void enqueue(int x) {

               Node newNode = new Node(x);
               if (tail == null) {
                   head = tail = newNode;
               } else {
                   tail.next = newNode;
                   tail = newNode;
               }
           }
```

```
        public int dequeue() {
            if (head == null) {
                throw new RuntimeException("Queue Underflow");
            }
            int val = head.data;
            head = head.next;
            if (head == null) {
                tail = null;
            }
            return val;
        }

        public int front() {
            if (head == null) {
                throw new RuntimeException("Queue is Empty");
            }
            return head.data;
        }
    }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(n)

175. Check for Balanced Parentheses

Understanding the Problem NEW

Given a string containing various types of brackets — `()`, `[]`, `{}` — check if every opening bracket has a matching, correctly-nested closing bracket. E.g. `"([{}])"` is balanced, but `"([)]"` is not (crossed, not nested).

Brute Force Approach NEW

Without a stack, you might try counting opening and closing brackets of each type separately and checking the counts match — but this misses ordering/nesting entirely (e.g. `"([)]"` has equal counts of each bracket type but is still invalid).

Why We Optimize NEW

A **stack** is exactly the right structure here because bracket matching is inherently LIFO: the *most recently opened* bracket must be the *next one closed*. Push every opening bracket onto the stack; on every closing bracket, check that it matches whatever is currently on top of the stack (popping it if so, or declaring the string unbalanced if not). If the stack is empty at the end, every bracket was properly matched. This correctly captures nesting order in a single O(n) pass, something a simple counting approach fundamentally cannot do.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if parentheses in string are balanced.

Visual Diagram

```
Input: "({[]})"
       Step 1: '(' push
       Step 2: '{' push
       Step 3: '[' push
       Step 4: ']' matches '[' → pop
       Step 5: '}' matches '{' → pop
       Step 6: ')' matches '(' → pop
       Result: Balanced ✓

       Input: "([)]"
```

```
Step 1: '(' push
Step 2: '[' push
Step 3: ')' doesn't match '[' → Unbalanced ✗

Input: "((()))"
Step 1: '(' push
Step 2: '(' push
Step 3: '(' push
Step 4: ')' matches '(' → pop
Step 5: ')' matches '(' → pop
End with '(' remaining → Unbalanced ✗
```

Solution Approach

Concept: Stack for opening brackets.

Algorithm

```
Algorithm IS_BALANCED(s):
           stack = Stack()
           For each char in s:
               If char in '({[':
                   stack.push(char)
               Else:
                   If stack.isEmpty(): Return false
                   top = stack.pop()
                   If not match(top, char): Return false
           Return stack.isEmpty()
```

Java Implementation

```
class Solution {
           public boolean isBalanced(String s) {
               Stack<Character> stack = new Stack<>();

                 for (char c : s.toCharArray()) {
                     if (c == '(' || c == '{' || c == '[') {
                         stack.push(c);
                     } else {
                         if (stack.isEmpty()) return false;
                         char top = stack.pop();
                         if (!isMatching(top, c)) return false;
                     }
                 }
                 return stack.isEmpty();

         }

         private boolean isMatching(char open, char close) {
             return (open == '(' && close == ')') ||
```

```
                    (open == '{' && close == '}') ||
                    (open == '[' && close == ']');
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

176. Implement Min Stack

Understanding the Problem NEW

Design a stack that supports the usual push/pop, plus a `getMin()` operation that returns the current minimum element in the stack — all in O(1) time.

Brute Force Approach NEW

You could scan the entire stack every time `getMin()` is called to find the current minimum — O(n) per call, which defeats the purpose if getMin() is called frequently.

Why We Optimize NEW

Maintain a **second stack** alongside the main one, which at every point tracks the minimum value *as of that point in the stack's history* — whenever you push a new value onto the main stack, you also push the smaller of (new value, current min) onto the min-stack; whenever you pop from the main stack, you also pop from the min-stack. This way, the top of the min-stack always instantly reflects the current minimum in O(1), because it's updated incrementally alongside every push/pop rather than recomputed from scratch on each query — trading a little extra memory (the second stack) for O(1) instead of O(n) queries.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement stack that supports getMin() in O(1).

Visual Diagram

```
Operations:
       push(5): stack=[5], minStack=[5] → min=5
       push(3): stack=[5,3], minStack=[5,3] → min=3
       push(8): stack=[5,3,8], minStack=[5,3] → min=3
       push(2): stack=[5,3,8,2], minStack=[5,3,2] → min=2
       pop(): stack=[5,3,8], minStack=[5,3] → min=3

       Alternative: Store only one stack with (value, min)
       Push: push (5,5), (3,3), (8,3), (2,2)
       Pop: pop (2,2) → min back to 3
```

Solution Approach

Concept: Maintain a separate min stack or store min with value.

Algorithm

```
Algorithm MIN_STACK:
           stack = Stack()
           minStack = Stack()

           push(x):
               stack.push(x)
               If minStack.isEmpty() or x <= minStack.peek():
                    minStack.push(x)

           pop():
               val = stack.pop()
               If val == minStack.peek():
                   minStack.pop()
               Return val

           getMin():
               Return minStack.peek()
```

Java Implementation

```
class MinStack {
           private Stack<Integer> stack;
           private Stack<Integer> minStack;

           MinStack() {
               stack = new Stack<>();
               minStack = new Stack<>();
           }

           public void push(int x) {
               stack.push(x);
               if (minStack.isEmpty() || x <= minStack.peek()) {
                   minStack.push(x);
               }
           }

           public void pop() {
               if (stack.isEmpty()) return;

              int val = stack.pop();
              if (val == minStack.peek()) {
                  minStack.pop();
              }
          }
```

```
        public int top() {
            return stack.peek();
        }

        public int getMin() {
            return minStack.peek();
        }
    }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(n)

177. Infix to Postfix Conversion

Understanding the Problem NEW

Convert an infix expression (the normal way we write math, like `a+b*c`, where operators sit between operands) into postfix notation (like `abc*+`, where operators come after their operands — this is what makes evaluation unambiguous without needing parentheses or precedence rules at evaluation time).

Brute Force Approach NEW

Without a stack, you might try to track operator precedence and parenthesization with nested recursive parsing, similar to writing a small parser — doable, but considerably more complex than necessary for this specific conversion.

Why We Optimize NEW

A single stack of operators does this cleanly: scan left to right, sending operands directly to the output; when you see an operator, **pop and output any operators currently on the stack that have equal or higher precedence** (since they must be applied first), then push the new operator; parentheses are handled by pushing '(' and popping back to it when ')' is seen. This is a clean single-pass, O(n) algorithm that handles precedence and parentheses uniformly through the stack, rather than requiring a full recursive expression parser.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert infix expression to postfix notation.

Visual Diagram

```
Input: a + b * c
       Output: a b c * +

       Step-by-step:
       a: output "a"
       +: stack [+]
       b: output "a b"
       *: stack [+, *]
```

```
c: output "a b c"
End: pop all → "a b c * +"

Input: (a + b) * c
Output: a b + c *

Step-by-step:
(: stack [(]
a: output "a"
+: stack [(, +]
b: output "a b"
): pop until ( → output "a b +"
*: stack [*]
c: output "a b + c"
End: pop * → "a b + c *"
```

Solution Approach

Concept: Stack with operator precedence.

Algorithm

```
Algorithm INFIX_TO_POSTFIX(exp):
           stack = Stack()
           result = ""
           precedence = {'+':1, '-':1, '*':2, '/':2, '^':3}

            For each char in exp:
                If char is operand:
                    result += char
                Else If char == '(':
                    stack.push(char)
                Else If char == ')':
                    While stack.peek() != '(':
                        result += stack.pop()
                    stack.pop() // remove '('
                Else:
                    While !stack.isEmpty() and stack.peek() != '(' and
                          precedence[stack.peek()] >= precedence[char]:
                        result += stack.pop()
                    stack.push(char)

             While !stack.isEmpty():
                 result += stack.pop()

          Return result
```

Java Implementation

```
class Solution {
          public String infixToPostfix(String exp) {
              Map<Character, Integer> precedence = new HashMap<>();
              precedence.put('+', 1);
              precedence.put('-', 1);
              precedence.put('*', 2);
              precedence.put('/', 2);
              precedence.put('^', 3);

              Stack<Character> stack = new Stack<>();
              StringBuilder result = new StringBuilder();

              for (char c : exp.toCharArray()) {
                  if (Character.isLetterOrDigit(c)) {
                      result.append(c);
                  } else if (c == '(') {
                      stack.push(c);
                  } else if (c == ')') {
                      while (!stack.isEmpty() && stack.peek() != '(') {
                          result.append(stack.pop());
                      }
                      stack.pop(); // Remove '('
                  } else { // Operator
                      while (!stack.isEmpty() && stack.peek() != '(' &&
                             precedence.getOrDefault(stack.peek(), 0) >=
precedence.get(c)) {
                          result.append(stack.pop());
                      }
                      stack.push(c);
                  }
              }

              while (!stack.isEmpty()) {
                  result.append(stack.pop());
              }
              return result.toString();
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

178. Prefix to Infix Conversion

Understanding the Problem NEW

Convert a prefix expression (operator before its operands, like `+a*bc`) into infix notation (the normal `a+b*c` style, with parentheses added as needed to make precedence explicit).

Brute Force Approach NEW

You could try to build a full expression tree first and then read it back out in-order with parentheses — a valid approach, but more machinery (an explicit tree structure) than a simple stack-based method needs.

Why We Optimize NEW

Since prefix means 'operator first, then its two operands,' scanning the expression from **right to left** and using a stack works naturally: when you see an operand, push it; when you see an operator, pop the two most recent items (which are its two operands, since you're scanning backward), combine them into a single string wrapped in parentheses with the operator in the middle, and push that combined string back onto the stack. By the end, the stack holds exactly one item — the fully parenthesized infix expression — all without building an explicit tree data structure.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert prefix expression to infix notation.

Visual Diagram

```
Input: * + A B C
       Output: (A + B) * C

       Step-by-step (read right to left):
       C: push C
       B: push B
       A: push A
       +: pop A, B → (A + B) → push
       *: pop (A+B), C → ((A+B) * C) → push
```

```
Input: - / A B * C D
Output: (A / B) - (C * D)

Read right to left:
D: push D
C: push C
*: pop C, D → (C*D) → push
B: push B
A: push A
/: pop A, B → (A/B) → push
-: pop (A/B), (C*D) → (A/B)-(C*D)
```

Solution Approach

Concept: Stack with right-to-left traversal.

Algorithm

```
Algorithm PREFIX_TO_INFIX(exp):
           stack = Stack()
           For i = exp.length-1 down to 0:
               If exp[i] is operand:
                   stack.push(exp[i])
               Else:
                   op1 = stack.pop()
                   op2 = stack.pop()
                   stack.push("(" + op1 + exp[i] + op2 + ")")
           Return stack.pop()
```

Java Implementation

```
class Solution {
           public String prefixToInfix(String exp) {
               Stack<String> stack = new Stack<>();

                 for (int i = exp.length() - 1; i >= 0; i--) {
                     char c = exp.charAt(i);

                     if (Character.isLetterOrDigit(c)) {
                         stack.push(String.valueOf(c));
                     } else {
                         String op1 = stack.pop();
                         String op2 = stack.pop();
                         stack.push("(" + op1 + c + op2 + ")");
                     }

              }
              return stack.pop();
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

179. Prefix to Postfix Conversion

Understanding the Problem NEW

Convert a prefix expression directly into postfix notation, without going through infix as an intermediate step.

Brute Force Approach NEW

You could convert prefix → infix (problem 178) and then infix → postfix (problem 177) as two separate stages — technically correct, but doing two full conversions when a direct one exists is unnecessary extra work.

Why We Optimize NEW

Using the same right-to-left scanning idea as problem 178: when you see an operand, push it; when you see an operator, pop its two operands and instead of wrapping them with parentheses and the operator in the middle (infix style), simply **concatenate the two operands followed by the operator** (postfix style) and push that combined string back. This direct conversion skips the intermediate infix representation entirely, doing in one pass what the two-stage approach would do in two.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert prefix expression to postfix notation.

Visual Diagram

```
Input: * + A B C
       Output: A B + C *

       Step-by-step (read right to left):
       C: push C
       B: push B
       A: push A
       +: pop A, B → A B + → push
       *: pop AB+, C → AB+ C * → push

       Input: - / A B * C D
       Output: A B / C D * -
```

```
Read right to left:
D: push D
C: push C
*: pop C, D → C D * → push
B: push B
A: push A
/: pop A, B → A B / → push
-: pop AB/, CD* → AB/ CD* - → push
```

Solution Approach

Concept: Stack with right-to-left traversal.

Algorithm

```
Algorithm PREFIX_TO_POSTFIX(exp):
           stack = Stack()
           For i = exp.length-1 down to 0:
               If exp[i] is operand:
                   stack.push(exp[i])
               Else:
                   op1 = stack.pop()
                   op2 = stack.pop()
                   stack.push(op1 + op2 + exp[i])
           Return stack.pop()
```

Java Implementation

```
class Solution {
           public String prefixToPostfix(String exp) {
               Stack<String> stack = new Stack<>();

                 for (int i = exp.length() - 1; i >= 0; i--) {
                     char c = exp.charAt(i);

                     if (Character.isLetterOrDigit(c)) {
                         stack.push(String.valueOf(c));
                     } else {
                         String op1 = stack.pop();
                         String op2 = stack.pop();
                         stack.push(op1 + op2 + c);
                     }

              }
              return stack.pop();
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

180. Postfix to Prefix Conversion

Understanding the Problem NEW

Convert a postfix expression (operands then operator, like `abc*+`) into prefix notation (`+a*bc`).

Brute Force Approach NEW

Similarly, you could convert postfix → infix (problem 181) then infix → prefix (problem 182) — two conversions instead of one direct pass.

Why We Optimize NEW

Postfix means operands come *before* their operator, so this time scan **left to right**: push operands as you see them; when you hit an operator, pop the two most recent operands (in postfix, they appear before the operator) and combine them as `operator + operand1 + operand2` (prefix order), pushing that combined string back. This mirrors problem 179's approach but scanning in the opposite direction, since postfix and prefix put their operators on opposite sides relative to the operands.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert postfix expression to prefix notation.

Visual Diagram

```
Input: A B C * +
       Output: + A * B C

       Step-by-step:
       A: push A
       B: push B
       C: push C
       *: pop C, B → * B C → push
       +: pop *BC, A → + A * B C

       Input: A B / C D * -
       Output: - / A B * C D
```

```
A: push A
B: push B
/: pop B, A → / A B → push
C: push C
D: push D
*: pop D, C → * C D → push
-: pop *CD, /AB → - /AB *CD
```

Solution Approach

Concept: Stack with left-to-right traversal.

Algorithm

```
Algorithm POSTFIX_TO_PREFIX(exp):
           stack = Stack()
           For each char in exp:
               If char is operand:
                   stack.push(char)
               Else:
                   op2 = stack.pop()
                   op1 = stack.pop()
                   stack.push(char + op1 + op2)
           Return stack.pop()
```

Java Implementation

```
class Solution {
           public String postfixToPrefix(String exp) {
               Stack<String> stack = new Stack<>();

                 for (char c : exp.toCharArray()) {
                     if (Character.isLetterOrDigit(c)) {
                         stack.push(String.valueOf(c));
                     } else {
                         String op2 = stack.pop();
                         String op1 = stack.pop();
                         stack.push(c + op1 + op2);
                     }
                 }
                 return stack.pop();
            }

     }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

181. Postfix to Infix Conversion

Understanding the Problem NEW

Convert a postfix expression into infix notation (with appropriate parentheses).

Brute Force Approach NEW

You could build a full expression tree from the postfix expression and then read it back in-order — valid, but more structure than needed.

Why We Optimize NEW

Scan left to right with a stack: push operands; when you hit an operator, pop the two most recent operands and combine them as `(operand1 operator operand2)` — parenthesized infix style — pushing the combined string back. Same left-to-right scanning direction as problem 180 (since postfix always processes operators after seeing both their operands), just building a parenthesized infix string instead of a prefix one.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert postfix expression to infix notation.

Visual Diagram

```
Input: A B C * +
       Output: A + (B * C)

       Step-by-step:
       A: push A
       B: push B
       C: push C
       *: pop C, B → (B * C) → push
       +: pop (B*C), A → A + (B*C) → push

       Input: A B / C D * -
       Output: (A / B) - (C * D)

       A: push A
       B: push B
       /: pop B, A → (A/B) → push
```

```
        C: push C
        D: push D
        *: pop D, C → (C*D) → push
        -: pop (C*D), (A/B) → (A/B) - (C*D)
```

Solution Approach

Concept: Stack with left-to-right traversal.

Algorithm

```
Algorithm POSTFIX_TO_INFIX(exp):
            stack = Stack()
            For each char in exp:
                If char is operand:
                    stack.push(char)
                Else:
                    op2 = stack.pop()
                    op1 = stack.pop()
                    stack.push("(" + op1 + char + op2 + ")")
            Return stack.pop()
```

Java Implementation

```
class Solution {
            public String postfixToInfix(String exp) {
                Stack<String> stack = new Stack<>();

                  for (char c : exp.toCharArray()) {
                      if (Character.isLetterOrDigit(c)) {
                          stack.push(String.valueOf(c));
                      } else {
                          String op2 = stack.pop();
                          String op1 = stack.pop();
                          stack.push("(" + op1 + c + op2 + ")");
                      }
                  }
                  return stack.pop();
             }

      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

182. Convert Infix To Prefix Notation

Understanding the Problem NEW

Convert an infix expression directly into prefix notation.

Brute Force Approach NEW

You might try to directly adapt the infix-to-postfix algorithm (problem 177) with modified precedence rules scanned from a different direction, but getting the details right without a clear method is error-prone.

Why We Optimize NEW

There's an elegant reduction here: **reverse the infix expression**, swap every '(' with ')' and vice versa (since reversing an expression flips its parenthesis structure), run the standard infix-to-postfix algorithm (problem 177) on this modified string, and then **reverse the resulting postfix string** to get the prefix result. This clever transformation reuses the already-solved infix-to-postfix algorithm as a subroutine, rather than deriving a brand new algorithm from scratch for prefix conversion — a good example of solving a new problem by reducing it to one you've already solved.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert infix expression to prefix notation.

Visual Diagram

```
Input: A + B * C
       Output: + A * B C

       Step 1: Reverse the string: C * B + A
       Step 2: Convert to postfix (with swapped precedence)
       Step 3: Reverse the result

       Detailed:
       Reverse: C * B + A
       Postfix conversion:
       C: push C
       *: stack [*]
```

```
        B: push B
        +: pop *, push + → C B *
        A: push A
        Pop + → C B * A +
        Reverse: + A * B C

        Input: (A + B) * C
        Output: * + A B C

        Reverse: C * ) B + A (
        Postfix: C B A + *
        Reverse: * + A B C
```

Solution Approach

Concept: Reverse, swap parentheses, convert to postfix, reverse.

Algorithm

```
Algorithm INFIX_TO_PREFIX(exp):
            Reverse exp
            Swap '(' and ')' in reversed string
            Convert to Postfix (with swapped precedence)
            Reverse result
            Return result
```

Java Implementation

```
class Solution {
            public String infixToPrefix(String exp) {
                // Reverse the expression
                StringBuilder reversed = new StringBuilder(exp).reverse();

                  // Swap parentheses
                  for (int i = 0; i < reversed.length(); i++) {
                      if (reversed.charAt(i) == '(') {
                          reversed.setCharAt(i, ')');
                      } else if (reversed.charAt(i) == ')') {
                          reversed.setCharAt(i, '(');
                      }
                  }

                  // Convert to postfix

               String postfix = infixToPostfix(reversed.toString());

               // Reverse result
               return new StringBuilder(postfix).reverse().toString();
           }
```

```
          private String infixToPostfix(String exp) {
              Map<Character, Integer> precedence = new HashMap<>();
              precedence.put('+', 1);
              precedence.put('-', 1);
              precedence.put('*', 2);
              precedence.put('/', 2);
              precedence.put('^', 3);

              Stack<Character> stack = new Stack<>();
              StringBuilder result = new StringBuilder();

              for (char c : exp.toCharArray()) {
                  if (Character.isLetterOrDigit(c)) {
                      result.append(c);
                  } else if (c == '(') {
                      stack.push(c);
                  } else if (c == ')') {
                      while (!stack.isEmpty() && stack.peek() != '(') {
                           result.append(stack.pop());
                      }
                      stack.pop();
                  } else {
                      while (!stack.isEmpty() && stack.peek() != '(' &&
                              precedence.getOrDefault(stack.peek(), 0) >=
precedence.get(c)) {
                           result.append(stack.pop());
                      }
                      stack.push(c);
                  }
              }

              while (!stack.isEmpty()) {
                  result.append(stack.pop());
              }
              return result.toString();
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

183. Next Greater Element

Understanding the Problem NEW

For every element in an array, find the next element to its right that is strictly greater than it (or -1/None if no such element exists). E.g. in `[4,5,2,10]` : NGE of 4 is 5, NGE of 5 is 10, NGE of 2 is 10, NGE of 10 is none.

Brute Force Approach NEW

For each element, scan forward through every element to its right until you find one that's bigger — $O(n^2)$ in the worst case, since you might rescan large portions of the array repeatedly from different starting points.

Why We Optimize NEW

Process the array from **right to left**, maintaining a stack that always holds a **decreasing** sequence of 'candidate' values (potential next-greater-elements for whatever comes before them). For each new element, pop off anything on the stack that's smaller or equal to it (since those values can never be the answer for anything earlier — the current element is a better, closer candidate), then whatever remains on top of the stack (if anything) is the current element's actual next greater element; finally push the current element itself onto the stack. Because each element is pushed and popped from the stack at most once across the whole process, the total work is O(n) rather than $O(n^2)$ — this is the foundational 'monotonic stack' pattern that powers nearly every remaining problem through 195.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the next greater element for each array element.

Visual Diagram

```
Array: [4, 5, 2, 10, 8]
       Next Greater: [5, 10, 10, -1, -1]

       Step-by-step (traverse from right):
       i=4: stack=[], NGE[4]=-1, push 8
       i=3: stack=[8], 10>8 → NGE[3]=-1, push 10
```

```
        i=2: stack=[10], 2<10 → NGE[2]=10, push 2
        i=1: stack=[2,10], 5>2 → pop, 5<10 → NGE[1]=10, push 5
        i=0: stack=[5,10], 4<5 → NGE[0]=5, push 4

        Array: [1, 3, 2, 4]
        NGE: [3, 4, 4, -1]

        Traverse from right:
        i=3: [] → -1, push 4
        i=2: [4] → 4, push 2
        i=1: [2,4] → 4, push 3
        i=0: [3,4] → 3
```

Solution Approach

Concept: Stack maintains decreasing order.

Algorithm

```
Algorithm NEXT_GREATER(arr):
            stack = Stack()
            result = new int[n]

            For i = n-1 down to 0:
                While !stack.isEmpty() and stack.peek() <= arr[i]:
                    stack.pop()
                result[i] = stack.isEmpty() ? -1 : stack.peek()
                stack.push(arr[i])

            Return result
```

Java Implementation

```
class Solution {
            public int[] nextGreaterElement(int[] arr) {
                int n = arr.length;
                int[] result = new int[n];
                Stack<Integer> stack = new Stack<>();

                  for (int i = n - 1; i >= 0; i--) {
                      while (!stack.isEmpty() && stack.peek() <= arr[i]) {
                          stack.pop();
                      }
                      result[i] = stack.isEmpty() ? -1 : stack.peek();
                      stack.push(arr[i]);
                  }

               return result;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

184. Next Greater Element 2

Understanding the Problem NEW

Same as Next Greater Element, but the array is **circular** — meaning after the last element, you wrap back around to the beginning when searching for a next greater element.

Brute Force Approach NEW

You could physically duplicate the array (concatenate it with itself) and run ordinary Next Greater Element on the doubled array — this works but processes 2n elements when a smaller adjustment suffices.

Why We Optimize NEW

Rather than actually duplicating the array in memory, simulate a 'virtual' doubled array by iterating index `i` from `2n-1` i % n to access the actual array — the monotonic stack technique from problem 183 applies completely unchanged otherwise. This achieves the same effect as physically doubling the array, but with O(1) extra indexing arithmetic instead of O(n) extra memory for an actual duplicated array.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find next greater element in circular array.

Visual Diagram

```
Array: [2, 10, 12, 1, 11]
       NGE Circular: [10, 12, -1, 11, 12]

       Traverse array twice (2n):
       i=9 (4): [11] → -1, push 11
       i=8 (3): [11] → 11, push 1
       i=7 (2): [1,11] → 11? Wait, 12>1, push 12
       i=6 (1): [12] → 12, push 10
       i=5 (0): [10,12] → 10? Actually 10>2, push 10
       i=4 (4): [10] → 10? Actually 11>10, wait...

       Let's trace properly:
```

```
Array: [2, 10, 12, 1, 11]
i=9 (4): stack=[], NGE[4]=-1, push 11
i=8 (3): stack=[11], 1<11 → NGE[3]=11, push 1
i=7 (2): stack=[1,11], 12>1, pop; 12>11, pop; empty → NGE[2]=-1, push 12
i=6 (1): stack=[12], 10<12 → NGE[1]=12, push 10
i=5 (0): stack=[10,12], 2<10 → NGE[0]=10, push 2
i=4 (4): stack=[2,10,12], 11>2, pop; 11>10, pop; 11<12 → NGE[4]=12
i=3 (3): stack=[11,12], 1<11 → NGE[3]=11
```

Solution Approach

Concept: Traverse twice using modulo.

Algorithm

```
Algorithm NEXT_GREATER_CIRCULAR(arr):
           n = arr.length
           result = new int[n]
           stack = Stack()
           Arrays.fill(result, -1)

            For i = 0 to 2*n-1:
                index = i % n
                While !stack.isEmpty() && arr[stack.peek()] < arr[index]:
                    result[stack.pop()] = arr[index]
                If i < n:
                    stack.push(index)

            Return result
```

Java Implementation

```
class Solution {
           public int[] nextGreaterElements(int[] nums) {
               int n = nums.length;
               int[] result = new int[n];
               Arrays.fill(result, -1);
               Stack<Integer> stack = new Stack<>();

                 for (int i = 0; i < 2 * n; i++) {
                     int index = i % n;
                     while (!stack.isEmpty() && nums[stack.peek()] < nums[index]) {
                         result[stack.pop()] = nums[index];

                    }
                    if (i < n) {
                        stack.push(index);
                    }
              }
              return result;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

185. Next Smaller Element

Understanding the Problem NEW

The mirror image of Next Greater Element: for every element, find the next element to its right that is strictly smaller than it.

Brute Force Approach NEW

Same naive option as before: for each element, scan forward until you find a smaller one — O(n²) worst case.

Why We Optimize NEW

Exactly the same monotonic stack technique as problem 183, with the comparison direction flipped: maintain a stack of an **increasing** sequence of candidates (instead of decreasing), popping anything larger or equal since it can never be a valid 'next smaller' answer for what comes before it. Recognizing that 'next smaller' is just 'next greater' with the inequality flipped — reusing the exact same algorithm shape — is the actual lesson here, not a new technique.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the next smaller element for each array element.

Visual Diagram

```
Array: [4, 5, 2, 10, 8]
       Next Smaller: [2, 2, -1, 8, -1]

       Traverse from right:
       i=4: stack=[], NSE[4]=-1, push 8
       i=3: stack=[8], 10>8 → NSE[3]=8, push 10
       i=2: stack=[10,8], 2<10, pop; 2<8, pop; empty → NSE[2]=-1, push 2
       i=1: stack=[2], 5>2 → NSE[1]=2, push 5
       i=0: stack=[5,2], 4<5, pop; 4>2 → NSE[0]=2, push 4
```

Solution Approach

Concept: Similar to NGE but with smaller condition.

Algorithm

```
Algorithm NEXT_SMALLER(arr):
           stack = Stack()
           result = new int[n]

           For i = n-1 down to 0:
               While !stack.isEmpty() && stack.peek() >= arr[i]:
                   stack.pop()
               result[i] = stack.isEmpty() ? -1 : stack.peek()
               stack.push(arr[i])

           Return result
```

Java Implementation

```
class Solution {
           public int[] nextSmallerElement(int[] arr) {
               int n = arr.length;
               int[] result = new int[n];
               Stack<Integer> stack = new Stack<>();

                 for (int i = n - 1; i >= 0; i--) {
                     while (!stack.isEmpty() && stack.peek() >= arr[i]) {
                         stack.pop();
                     }
                     result[i] = stack.isEmpty() ? -1 : stack.peek();
                     stack.push(arr[i]);
                 }
                 return result;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

186. Number of NGEs to the Right

Understanding the Problem NEW

Instead of finding just the *first* next-greater-element, count *how many* elements to the right of each position are greater than it.

Brute Force Approach NEW

For each element, scan every element to its right and count how many are bigger — O(n²).

Why We Optimize NEW

This can also be solved with a monotonic stack, but tracking counts alongside values: rather than storing just candidate values on the stack, you also track how many elements have already been 'accounted for' relative to each candidate, incrementing counts as elements get popped in a way that reflects how many subsequent elements were bigger. This still processes each element a bounded number of times overall, keeping it at O(n) instead of O(n²), extending the same monotonic-stack contribution-counting idea used elsewhere in this chapter (like Sum of Subarray Minimums, problem 188).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count how many next greater elements exist to the right.

Visual Diagram

```
Array: [3, 4, 2, 7, 5, 8, 10, 6]
       For each element, count NGEs:

       i=0 (3): elements >3: 4,7,5,8,10,6 → 6
       i=1 (4): elements >4: 7,5,8,10,6 → 5
       i=2 (2): elements >2: 7,5,8,10,6 → 5
       i=3 (7): elements >7: 8,10 → 2
       i=4 (5): elements >5: 8,10,6 → 3
       i=5 (8): elements >8: 10 → 1
       i=6 (10): elements >10: none → 0
       i=7 (6): elements >6: none → 0
```

Solution Approach

Concept: Count greater elements to right.

Algorithm

```
Algorithm COUNT_NGE(arr):
           n = arr.length
           result = new int[n]

           For i = 0 to n-1:
               count = 0
               For j = i+1 to n-1:
                   If arr[j] > arr[i]: count++
               result[i] = count

           Return result
```

Java Implementation

```
class Solution {
           public int[] countNGE(int[] arr) {
               int n = arr.length;
               int[] result = new int[n];

                 for (int i = 0; i < n; i++) {
                     int count = 0;
                     for (int j = i + 1; j < n; j++) {
                         if (arr[j] > arr[i]) {
                             count++;
                         }
                     }
                     result[i] = count;
                 }
                 return result;
           }
      }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n)

187. Trapping Rainwater

Understanding the Problem NEW

Given a histogram of wall heights, calculate how much rainwater can be trapped between the walls after it rains. E.g. heights `[0,1,0,2,1,0,1,3,2,1,2,1]` traps 6 units of water.

Brute Force Approach NEW

For every position, scan left to find the tallest wall to its left and scan right to find the tallest wall to its right, then the water trapped at that position is `min(leftMax, rightMax) - height[i]` — correct, but this recomputes the left-max and right-max scans for every single position independently, giving O(n²).

Why We Optimize NEW

You don't need to rescan for every position — precompute the running maximum height **from the left** and **from the right** for every position in two single O(n) passes (storing them in arrays), then combine them in a third O(n) pass using the same formula. This turns O(n²) repeated scanning into three linear passes, O(n) total — reusing already-computed prefix/suffix maximums instead of recalculating them per position. (An even more memory-efficient two-pointer version exists that computes this with O(1) extra space instead of the two extra arrays, by cleverly advancing whichever side currently has the smaller max — worth knowing as a further refinement.)

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Calculate how much rainwater can be trapped.

Visual Diagram

```
Array: [0,1,0,2,1,0,1,3,2,1,2,1]
       Water trapped = 6 units

       Visual:
                ■
            ■   ■ ■     ■
          ■   ■ ■ ■ ■ ■ ■
```

```
      ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■
      0 1 0 2 1 0 1 3 2 1 2 1

      Two-pointer approach:
      left=0, right=11, leftMax=0, rightMax=0, water=0

      height[0]=0 < height[11]=1:
        leftMax=0, water+=0, left++ → 1
      height[1]=1 < height[11]=1:
        leftMax=1, water+=0, left++ → 2
      height[2]=0 < height[11]=1:
        leftMax=1, water+=1-0=1, left++ → 3
      height[3]=2 > height[11]=1:
        rightMax=1, water+=0, right-- → 10
      ...
      Total water = 6
```

Solution Approach

Concept: Two pointers with max heights.

Algorithm

```
Algorithm TRAP_RAINWATER(height):
          left = 0, right = n-1
          leftMax = 0, rightMax = 0
          water = 0

          While left < right:
              If height[left] < height[right]:
                  If height[left] >= leftMax:
                      leftMax = height[left]
                  Else:
                      water += leftMax - height[left]
                  left++
              Else:
                  If height[right] >= rightMax:
                      rightMax = height[right]
                  Else:
                      water += rightMax - height[right]
                  right--

          Return water
```

Java Implementation

```
class Solution {
          public int trap(int[] height) {

             int left = 0, right = height.length - 1;
```

```
            int leftMax = 0, rightMax = 0;
            int water = 0;

            while (left < right) {
                if (height[left] < height[right]) {
                    if (height[left] >= leftMax) {
                        leftMax = height[left];
                    } else {
                        water += leftMax - height[left];
                    }
                    left++;
                } else {
                    if (height[right] >= rightMax) {
                        rightMax = height[right];
                    } else {
                        water += rightMax - height[right];
                    }
                    right--;
                }
            }
            return water;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

188. Sum of Subarray Minimums

Understanding the Problem NEW

For every possible subarray of the array, find its minimum element, and sum up all of these minimums across every subarray. Directly computing this for every subarray individually is expensive, since there are $O(n^2)$ subarrays.

Brute Force Approach NEW

Generate every subarray, find its minimum (an O(n) scan per subarray in the naive case), and sum them all — $O(n^3)$ total, or $O(n^2)$ if you at least track the running minimum incrementally as you extend each subarray rather than rescanning from scratch.

Why We Optimize NEW

Instead of thinking 'what is the minimum of each subarray,' flip the question around: **for each individual element, in how many subarrays is it the minimum?** An element's contribution to the total sum is (its value) × (the number of subarrays where it's the minimum). Using a monotonic stack, you can find, for each element, how far left and right it extends as 'the current minimum' (i.e. the distance to the nearest smaller element on each side) — multiplying those two distances tells you exactly how many subarrays have this element as their minimum. Summing (value × count) for every element gives the total in O(n), using the same 'contribution technique' that also solves Sum of Subarray Ranges (problem 190) — a much smarter reframing than directly computing every subarray's minimum.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find sum of minimums of all subarrays.

Visual Diagram

```
Array: [3, 1, 2, 4]

       All subarrays and minimums:
       [3] → 3
       [3,1] → 1
```

```
[3,1,2] → 1
[3,1,2,4] → 1
[1] → 1
[1,2] → 1
[1,2,4] → 1
[2] → 2
[2,4] → 2
[4] → 4

Sum = 3+1+1+1+1+1+1+2+2+4 = 17

Using Monotonic Stack:
For each element, find:
- Previous smaller (strict)
- Next smaller (non-strict)

[3, 1, 2, 4]
For 3: prev=-1, next=1 → (1-(-1))*(1-1)=2*0=0? Wrong

Actually: contribution = arr[i] * (i - prevSmaller) * (nextSmaller - i)
```

Solution Approach

Concept: Monotonic stack for contribution.

Algorithm

```
Algorithm SUM_SUBARRAY_MIN(arr):
           n = arr.length
           prevSmaller = new int[n]
           nextSmaller = new int[n]
           stack = Stack()

           // Previous smaller
           For i = 0 to n-1:
               While !stack.isEmpty() && arr[stack.peek()] > arr[i]:
                   stack.pop()
               prevSmaller[i] = stack.isEmpty() ? -1 : stack.peek()
               stack.push(i)

           stack.clear()
           // Next smaller
           For i = n-1 down to 0:
               While !stack.isEmpty() && arr[stack.peek()] >= arr[i]:
                   stack.pop()
               nextSmaller[i] = stack.isEmpty() ? n : stack.peek()
               stack.push(i)

           sum = 0
           For i = 0 to n-1:
```

```
            left = i - prevSmaller[i]
            right = nextSmaller[i] - i
            sum += arr[i] * left * right

        Return sum
```

Java Implementation

```
class Solution {
        public int sumSubarrayMins(int[] arr) {
            int n = arr.length;
            int[] prevSmaller = new int[n];
            int[] nextSmaller = new int[n];
            Stack<Integer> stack = new Stack<>();
            long sum = 0;
            int MOD = 1_000_000_007;

            // Previous smaller
            for (int i = 0; i < n; i++) {
                while (!stack.isEmpty() && arr[stack.peek()] > arr[i]) {
                    stack.pop();
                }
                prevSmaller[i] = stack.isEmpty() ? -1 : stack.peek();
                stack.push(i);
            }

            stack.clear();

            // Next smaller
            for (int i = n - 1; i >= 0; i--) {
                while (!stack.isEmpty() && arr[stack.peek()] >= arr[i]) {
                    stack.pop();
                }
                nextSmaller[i] = stack.isEmpty() ? n : stack.peek();
                stack.push(i);
            }

            for (int i = 0; i < n; i++) {
                long left = i - prevSmaller[i];
                long right = nextSmaller[i] - i;
                sum = (sum + arr[i] * left % MOD * right) % MOD;
            }
            return (int) sum;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

189. Asteroid Collision

Understanding the Problem NEW

Simulate a row of asteroids moving left or right (given as an array of signed integers, where sign indicates direction and magnitude indicates size); when two asteroids moving toward each other collide, the smaller one is destroyed (or both, if equal size), and larger destroys smaller. Determine the final state after all collisions resolve.

Brute Force Approach NEW

Simulate collisions with nested loops, re-scanning the array after each collision to check for new adjacent collisions that the resolution may have created — potentially $O(n^2)$ since each collision can trigger a re-scan.

Why We Optimize NEW

A **stack** naturally models this: process asteroids left to right, and whenever the current asteroid is moving left *and* the top of the stack is moving right (a genuine collision setup), resolve the collision by comparing sizes — possibly popping the stack repeatedly if the current asteroid keeps destroying smaller ones above it, or stopping early if it gets destroyed itself. Because each asteroid is pushed and popped from the stack at most once over the whole simulation, this achieves O(n) total, rather than repeatedly re-scanning the array after each individual collision.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Simulate asteroid collisions.

Visual Diagram

```
Input: [5, 10, -5]
       Output: [5, 10]

       Step 1: 5 → stack [5]
       Step 2: 10 → stack [5,10]
       Step 3: -5 → moving left, collides with 10
       10 > 5 → -5 destroyed, stack [5,10]
```

```
Input: [10, 2, -5]
Output: [10]

Step 1: 10 → [10]
Step 2: 2 → [10,2]
Step 3: -5 → collides with 2
2 < 5 → 2 destroyed, continue with 10
10 > 5 → -5 destroyed, stack [10]

Input: [-5, 10, -5]
Output: [-5, 10]

Step 1: -5 → [-5]
Step 2: 10 → [-5,10]
Step 3: -5 → collides with 10
10 > 5 → -5 destroyed, stack [-5,10]

Input: [8, -8]
Output: []
Step 1: 8 → [8]
Step 2: -8 → collides, equal → both destroyed, []
```

Solution Approach

Concept: Stack with collision handling.

Algorithm

```
Algorithm ASTEROID_COLLISION(asteroids):
           stack = Stack()

            For asteroid in asteroids:
                If asteroid > 0:
                    stack.push(asteroid)
                Else:
                    While !stack.isEmpty() && stack.peek() > 0 && stack.peek() < -
asteroid:
                        stack.pop()
                    If stack.isEmpty() || stack.peek() < 0:
                        stack.push(asteroid)
                    Else If stack.peek() == -asteroid:
                        stack.pop()

            Return stack to array
```

Java Implementation

```
class Solution {

          public int[] asteroidCollision(int[] asteroids) {
```

```
            Stack<Integer> stack = new Stack<>();

            for (int asteroid : asteroids) {
                if (asteroid > 0) {
                    stack.push(asteroid);
                } else {
                    while (!stack.isEmpty() && stack.peek() > 0 && stack.peek() <
-asteroid) {
                         stack.pop();
                    }
                    if (stack.isEmpty() || stack.peek() < 0) {
                         stack.push(asteroid);
                    } else if (stack.peek() == -asteroid) {
                         stack.pop();
                    }
                }
            }

            int[] result = new int[stack.size()];
            for (int i = stack.size() - 1; i >= 0; i--) {
                result[i] = stack.pop();
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

190. Sum of Subarray Ranges

Understanding the Problem NEW

For every subarray, compute its range (max minus min), and sum this value across all subarrays. This combines the ideas of Sum of Subarray Minimums (188) and its mirror image, Sum of Subarray Maximums.

Brute Force Approach NEW

Generate every subarray, compute its max and min directly (an O(n) scan each), sum the ranges — $O(n^3)$, or $O(n^2)$ with incremental tracking.

Why We Optimize NEW

Exactly the same contribution-based reframing as problem 188, applied twice: compute the total sum of subarray **minimums** using a monotonic stack (as in 188), separately compute the total sum of subarray **maximums** using the mirrored monotonic stack technique, and the answer is simply (sum of maximums) minus (sum of minimums) — since range = max - min for every subarray, and sums distribute over subtraction. This reuses the exact same O(n) contribution technique twice rather than inventing a new approach for 'range' specifically.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find sum of (max - min) for all subarrays.

Visual Diagram

```
Array: [1, 2, 3]
       Subarrays and ranges:
       [1] → 0
       [1,2] → 1
       [1,2,3] → 2
       [2] → 0
       [2,3] → 1
       [3] → 0
       Sum = 4

       Method: Sum(max) - Sum(min)
```

```
       Sum(max) for [1,2,3]:
       Using monotonic stack:
       Next greater: [1,2,3] → [1,-1,-1]
       Previous greater: [-1,-1,1]

       Sum of ranges = Sum(max contributions) - Sum(min contributions)
```

Solution Approach

Concept: Use contribution of each element.

Algorithm

```
Algorithm SUM_SUBARRAY_RANGE(arr):
           return sumMax(arr) - sumMin(arr)

       Algorithm sumMax(arr):
           prevGreater = findPrevGreater(arr)
           nextGreater = findNextGreater(arr)
           sum = 0
           For i = 0 to n-1:
               left = i - prevGreater[i]
               right = nextGreater[i] - i
               sum += arr[i] * left * right
           Return sum
```

Java Implementation

```
class Solution {
           public long subArrayRanges(int[] nums) {
               return sumMax(nums) - sumMin(nums);
           }

           private long sumMax(int[] nums) {
               int n = nums.length;
               long sum = 0;
               Stack<Integer> stack = new Stack<>();
               int[] prevGreater = new int[n];
               int[] nextGreater = new int[n];

                 for (int i = 0; i < n; i++) {
                     while (!stack.isEmpty() && nums[stack.peek()] < nums[i]) {
                         stack.pop();
                     }

                  prevGreater[i] = stack.isEmpty() ? -1 : stack.peek();
                  stack.push(i);
              }

              stack.clear();
```

```
            for (int i = n - 1; i >= 0; i--) {
                while (!stack.isEmpty() && nums[stack.peek()] <= nums[i]) {
                    stack.pop();
                }
                nextGreater[i] = stack.isEmpty() ? n : stack.peek();
                stack.push(i);
            }

            for (int i = 0; i < n; i++) {
                sum += (long) nums[i] * (i - prevGreater[i]) * (nextGreater[i] -
i);
            }
            return sum;
        }

        private long sumMin(int[] nums) {
            int n = nums.length;
            long sum = 0;
            Stack<Integer> stack = new Stack<>();
            int[] prevSmaller = new int[n];
            int[] nextSmaller = new int[n];

            for (int i = 0; i < n; i++) {
                while (!stack.isEmpty() && nums[stack.peek()] > nums[i]) {
                    stack.pop();
                }
                prevSmaller[i] = stack.isEmpty() ? -1 : stack.peek();
                stack.push(i);
            }

            stack.clear();
            for (int i = n - 1; i >= 0; i--) {
                while (!stack.isEmpty() && nums[stack.peek()] >= nums[i]) {
                    stack.pop();
                }
                nextSmaller[i] = stack.isEmpty() ? n : stack.peek();
                stack.push(i);
            }

            for (int i = 0; i < n; i++) {
                sum += (long) nums[i] * (i - prevSmaller[i]) * (nextSmaller[i] -
i);
            }
            return sum;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

191. Remove K Digits

Understanding the Problem NEW

Given a number (as a string of digits) and a count K, remove exactly K digits so that the remaining digits (kept in their original order) form the smallest possible number.

Brute Force Approach NEW

Try every possible way of choosing which K digits to remove (there are C(n,K) ways) and check which resulting number is smallest — combinatorially expensive for anything but tiny inputs.

Why We Optimize NEW

A greedy, stack-based approach works: scan digits left to right, and whenever the current digit is **smaller** than the digit on top of the stack, pop the stack (removing that larger, earlier digit — using up one of your K removals) as long as you still have removals available, since removing a larger digit earlier always makes the resulting number smaller. This greedily builds an increasing sequence on the stack (removing 'bumps' that make the number larger than necessary), achieving the optimal result in a single O(n) pass instead of exploring every combination of removals.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Remove k digits to make smallest number.

Visual Diagram

```
Input: "1432219", k=3
       Output: "1219"

       Step-by-step:
       1: stack [1]
       4: stack [1,4]
       3: 3<4 → pop, stack [1], push 3 → [1,3]
       2: 2<3 → pop, stack [1], push 2 → [1,2]
       2: stack [1,2,2]
       1: 1<2 → pop, [1,2]; 1<2 → pop, [1]; push 1 → [1,1]
```

```
        9: stack [1,1,9]
        Remove last 1 → "1219"

        Input: "10200", k=1
        Output: "200"

        Step 1: 1 → [1]
        Step 2: 0<1 → pop, [0]
        Step 3: 2 → [0,2]
        Step 4: 0<2 → pop, [0,0]
        Step 5: 0 → [0,0,0]
        After removing k digits: "200" (remove leading zeros)
```

Solution Approach

Concept: Stack with monotonic increasing order.

Algorithm

```
Algorithm REMOVE_K_DIGITS(num, k):
           stack = Stack()

           For digit in num:
               While !stack.isEmpty() && k > 0 && stack.peek() > digit:
                   stack.pop()
                   k--
               stack.push(digit)

           While k > 0:
               stack.pop()
               k--

           // Remove leading zeros
           result = ""
           While !stack.isEmpty():
               result = stack.pop() + result

           // Remove leading zeros
           i = 0
           While i < result.length() && result.charAt(i) == '0':
               i++

           result = result.substring(i)
           Return result.isEmpty() ? "0" : result
```

Java Implementation

```
class Solution {
          public String removeKdigits(String num, int k) {
              Stack<Character> stack = new Stack<>();
```

```
            for (char digit : num.toCharArray()) {
                while (!stack.isEmpty() && k > 0 && stack.peek() > digit) {
                    stack.pop();
                    k--;
                }
                stack.push(digit);
            }

            while (k > 0 && !stack.isEmpty()) {
                stack.pop();
                k--;
            }

            StringBuilder result = new StringBuilder();
            while (!stack.isEmpty()) {
                result.append(stack.pop());
            }
            result.reverse();

            // Remove leading zeros
            int i = 0;
            while (i < result.length() && result.charAt(i) == '0') {
                i++;
            }
            result.delete(0, i);

            return result.length() == 0 ? "0" : result.toString();
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

192. Largest Rectangle in Histogram

Understanding the Problem NEW

Given a histogram (bar heights), find the area of the largest rectangle that can be formed using consecutive bars (the rectangle's height is limited by the shortest bar it spans).

Brute Force Approach NEW

For every pair of left and right boundaries, find the minimum height in that range and compute the area — O(n²) or O(n³) depending on how the minimum is found each time.

Why We Optimize NEW

For each bar, the largest rectangle *using that bar's height* extends as far left and right as possible while staying ≥ that bar's height — which is exactly 'how far until the nearest smaller bar on each side,' the same question a monotonic stack answers in O(n) (same core idea as problem 188's 'how far does this element extend as the minimum'). Compute the nearest-smaller-to-the-left and nearest-smaller-to-the-right for every bar using two monotonic stack passes, then the area for each bar is `height[i] × (right_boundary - left_boundary - 1)`, taking the maximum over all bars — O(n) total instead of O(n²).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the largest rectangle area in histogram.

Visual Diagram

```
Histogram: [2, 1, 5, 6, 2, 3]
       Largest rectangle = 10

       Visual:
            ■
          ■ ■
        ■ ■ ■
        ■ ■ ■   ■
      ■ ■ ■ ■ ■ ■
      2 1 5 6 2 3
```

```
Rectangle heights:
2: area = 2*1=2
1: area = 1*6=6
5: area = 5*2=10 (using height 5, width 2)
6: area = 6*1=6
2: area = 2*3=6
3: area = 3*1=3

Using Monotonic Stack:
For each bar, find previous smaller and next smaller
i=0 (2): prev=-1, next=1 → area=2*(1-(-1)-1)=2
i=1 (1): prev=-1, next=6 → area=1*6=6
i=2 (5): prev=1, next=3 → area=5*(3-1-1)=5
i=3 (6): prev=2, next=4 → area=6*(4-2-1)=6
i=4 (2): prev=1, next=6 → area=2*(6-1-1)=8
i=5 (3): prev=4, next=6 → area=3*(6-4-1)=3
Max = 10
```

Solution Approach

Concept: Monotonic stack for previous/next smaller.

Algorithm

```
Algorithm LARGEST_RECTANGLE(heights):
           stack = Stack()
           maxArea = 0
           n = heights.length

           For i = 0 to n:
               While !stack.isEmpty() && (i == n || heights[stack.peek()] >
heights[i]):
                   height = heights[stack.pop()]
                   width = stack.isEmpty() ? i : i - stack.peek() - 1
                   maxArea = Math.max(maxArea, height * width)
               stack.push(i)

           Return maxArea
```

Java Implementation

```
class Solution {
           public int largestRectangleArea(int[] heights) {
               Stack<Integer> stack = new Stack<>();
               int maxArea = 0;

              int n = heights.length;

              for (int i = 0; i <= n; i++) {
```

```
                int currentHeight = (i == n) ? 0 : heights[i];

                while (!stack.isEmpty() && heights[stack.peek()] > currentHeight)
{
                    int height = heights[stack.pop()];
                    int width = stack.isEmpty() ? i : i - stack.peek() - 1;
                    maxArea = Math.max(maxArea, height * width);
                }
                stack.push(i);
            }
            return maxArea;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

193. Maximal Rectangles

Understanding the Problem NEW

Given a binary matrix (containing only 0s and 1s), find the area of the largest rectangle made entirely of 1s.

Brute Force Approach NEW

Check every possible rectangle (every pair of top-left and bottom-right corners) and verify all cells inside are 1 — extremely expensive, O((rows×cols)²) or worse.

Why We Optimize NEW

This reduces directly to the Largest Rectangle in Histogram problem (192), applied **row by row**: treat each row as the 'ground level' of a histogram, where the height of each column at that row is how many consecutive 1s are stacked directly above it (including the current row) — updating these running heights incrementally as you move down each row (extending the count if the cell is 1, resetting to 0 if it's 0). For each row's resulting histogram, run the O(cols) Largest Rectangle in Histogram algorithm, and track the maximum area seen across all rows — turning a 2D rectangle-search problem into rows × (an O(cols) 1D problem you've already solved), giving O(rows × cols) total.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find largest rectangle of 1s in binary matrix.

Visual Diagram

```
Matrix:
       1 0 1 0 0
       1 0 1 1 1
       1 1 1 1 1
       1 0 0 1 0

       Largest rectangle: area 6 (rows 1-2, cols 2-4)

       Step-by-step:
       Convert each row to histogram heights:
```

```
Row 0: [1,0,1,0,0] → histogram [1,0,1,0,0]
Row 1: [1,0,1,1,1] → [2,0,2,1,1]
Row 2: [1,1,1,1,1] → [3,1,3,2,2]
Row 3: [1,0,0,1,0] → [4,0,0,3,0]

For each row, find largest rectangle in histogram:
Row 0: max = 1
Row 1: max = 2 (height 2, width 1)
Row 2: max = 6 (height 2, width 3)
Row 3: max = 4
Result = 6
```

Solution Approach

Concept: Convert to histogram and use largest rectangle.

Algorithm

```
Algorithm MAXIMAL_RECTANGLE(matrix):
           If matrix empty: Return 0
           m = rows, n = cols
           heights = new int[n]
           maxArea = 0

            For i = 0 to m-1:
                For j = 0 to n-1:
                    If matrix[i][j] == '1':
                        heights[j]++
                    Else:
                        heights[j] = 0
                maxArea = Math.max(maxArea, largestRectangleArea(heights))

            Return maxArea
```

Java Implementation

```
class Solution {
           public int maximalRectangle(char[][] matrix) {
               if (matrix.length == 0) return 0;

                 int rows = matrix.length, cols = matrix[0].length;
                 int[] heights = new int[cols];
                 int maxArea = 0;
                 for (int i = 0; i < rows; i++) {

                  for (int j = 0; j < cols; j++) {
                      if (matrix[i][j] == '1') {
                          heights[j]++;
                      } else {
                          heights[j] = 0;
```

```
                    }
                }
                maxArea = Math.max(maxArea, largestRectangleArea(heights));
            }
            return maxArea;
        }

        private int largestRectangleArea(int[] heights) {
            Stack<Integer> stack = new Stack<>();
            int maxArea = 0;
            int n = heights.length;

            for (int i = 0; i <= n; i++) {
                int currentHeight = (i == n) ? 0 : heights[i];

                while (!stack.isEmpty() && heights[stack.peek()] > currentHeight)
{
                    int height = heights[stack.pop()];
                    int width = stack.isEmpty() ? i : i - stack.peek() - 1;
                    maxArea = Math.max(maxArea, height * width);
                }
                stack.push(i);
            }
            return maxArea;
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(n)

194. Sliding Window Maximum

Understanding the Problem NEW

Given an array and a window size K, find the maximum value within every sliding window of size K as it moves across the array from left to right.

Brute Force Approach NEW

For every window position, scan all K elements within it to find the max — O(n×K) total, recomputing the max from scratch for every single window even though consecutive windows overlap heavily.

Why We Optimize NEW

Use a **deque (double-ended queue)** that maintains a decreasing sequence of candidate indices: before adding a new index, remove from the back any indices whose values are smaller than the new one (since they can never be the max while the new, larger element is still in the window); also remove from the front any index that has fallen outside the current window's range. The front of the deque always holds the index of the current window's maximum. Because each index is added and removed from the deque at most once over the whole array, this achieves O(n) total, instead of O(n×K) — reusing the 'maintain a monotonic sequence, discard now-useless candidates' idea from the monotonic stack problems, but applied to a sliding window with a deque instead of a stack.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find maximum in each sliding window of size k.

Visual Diagram

```
Array: [1,3,-1,-3,5,3,6,7], k=3
       Output: [3,3,5,5,6,7]

       Step-by-step:
       Window 1: [1,3,-1] → max=3
       Window 2: [3,-1,-3] → max=3
       Window 3: [-1,-3,5] → max=5
       Window 4: [-3,5,3] → max=5
```

```
Window 5: [5,3,6] → max=6
Window 6: [3,6,7] → max=7

Using Deque:
i=0: deque=[0], arr[0]=1
i=1: deque=[1], arr[1]=3
i=2: deque=[1,2], arr[1]=3, arr[2]=-1 → max=3
i=3: remove 1, deque=[2,3], arr[3]=-3 → max=3
i=4: deque=[4], arr[4]=5 → max=5
```

Solution Approach

Concept: Deque maintains decreasing order.

Algorithm

```
Algorithm SLIDING_WINDOW_MAX(arr, k):
           deque = Deque()
           result = []

           For i = 0 to n-1:
               // Remove out of window
               While !deque.isEmpty() && deque.peekFirst() <= i-k:
                   deque.pollFirst()

                 // Remove smaller elements
                 While !deque.isEmpty() && arr[deque.peekLast()] <= arr[i]:
                     deque.pollLast()

                 deque.offerLast(i)

                 If i >= k-1:
                     result.add(arr[deque.peekFirst()])

           Return result
```

Java Implementation

```
class Solution {
           public int[] maxSlidingWindow(int[] nums, int k) {
               int n = nums.length;
               int[] result = new int[n - k + 1];
               Deque<Integer> deque = new ArrayDeque<>();
               for (int i = 0; i < n; i++) {

                  // Remove elements outside window
                  while (!deque.isEmpty() && deque.peekFirst() <= i - k) {
                      deque.pollFirst();
                  }
```

```
                // Remove smaller elements
                while (!deque.isEmpty() && nums[deque.peekLast()] <= nums[i]) {
                    deque.pollLast();
                }

                deque.offerLast(i);

                if (i >= k - 1) {
                    result[i - k + 1] = nums[deque.peekFirst()];
                }
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(k)

195. Stock Span Problem

Understanding the Problem NEW

Given a sequence of daily stock prices, compute the 'span' for each day — the number of consecutive days (including today) going backward where the price was less than or equal to today's price.

Brute Force Approach NEW

For each day, scan backward day by day until you find a higher price, counting how far back you went — O(n²) in the worst case (e.g. steadily increasing prices).

Why We Optimize NEW

This is Next Greater Element (problem 183) applied backward: maintain a monotonic stack of (price, span-so-far) pairs; for each new day, pop off any days from the stack whose price is ≤ today's price, **accumulating their spans** into today's span (since if a previous day's price was smaller, everything that day's span already covered is also covered by today going back further), then push today's price and computed span. Because each day is pushed and popped at most once, this achieves O(n) instead of O(n²) — a direct, if reframed, application of the monotonic stack idea used throughout this chapter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Calculate stock span (consecutive days with price <= today).

Visual Diagram

```
Prices: [100, 80, 60, 70, 60, 75, 85]
       Span:   [1,   1, 1, 2, 1, 4, 6]

       Step-by-step:
       100 → span=1
       80 → span=1
       60 → span=1
       70 → 70>60, span=2
       60 → span=1
       75 → 75>60, >70, >80? No, span=4 (60,70,60)
       85 → 85>75, >80, >100? No, span=6
```

```
Using Stack:
i=0: stack=[0], span[0]=1
i=1: 80<100 → span=1, stack=[0,1]
i=2: 60<80 → span=1, stack=[0,1,2]
i=3: 70>60 → pop 2, stack=[0,1]; 70<80 → span=3-1=2
i=4: 60<70 → span=1
i=5: 75>60 → pop 4, >70 → pop 3, <80 → span=5-1=4
i=6: 85>75 → pop 5, >80 → pop 1, <100 → span=6-0=6
```

Solution Approach

Concept: Stack for previous greater element.

Algorithm

```
Algorithm STOCK_SPAN(prices):
           stack = Stack()
           span = new int[n]

           For i = 0 to n-1:
               While !stack.isEmpty() && prices[stack.peek()] <= prices[i]:
                   stack.pop()
               span[i] = stack.isEmpty() ? i+1 : i - stack.peek()
               stack.push(i)

           Return span
```

Java Implementation

```
class Solution {
           public int[] stockSpan(int[] prices) {
               int n = prices.length;
               int[] span = new int[n];
               Stack<Integer> stack = new Stack<>();

                 for (int i = 0; i < n; i++) {
                     while (!stack.isEmpty() && prices[stack.peek()] <= prices[i])
{
                         stack.pop();
                     }
                     span[i] = stack.isEmpty() ? i + 1 : i - stack.peek();
                     stack.push(i);
                 }
                 return span;

           }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

196. The Celebrity Problem

Understanding the Problem NEW

Among a group of N people, a 'celebrity' is someone who is known by everyone else but knows no one else themselves. Given a way to check 'does person A know person B' for any pair, find the celebrity (if one exists) using as few such checks as possible.

Brute Force Approach NEW

Check every pair of people to build a complete 'who knows who' picture, then look for the one person matching the celebrity definition — $O(n^2)$ checks, which is correct but doesn't exploit any structure in the celebrity property itself.

Why We Optimize NEW

The key insight: if person A knows person B, then A definitely *isn't* the celebrity (celebrities know no one) — so you can immediately discard A as a candidate. Using two pointers scanning through candidates and repeatedly eliminating whichever one 'knows' the other, you can narrow down to a **single remaining candidate** in O(n) checks (each check eliminates exactly one candidate), and then do one final O(n) verification pass to confirm that sole remaining candidate is actually known by everyone and knows no one. This 'elimination' strategy needs far fewer checks than examining every pair, dropping from $O(n^2)$ to O(n).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the celebrity in a group of people.

Visual Diagram

```
Matrix where knows[a][b] = true if a knows b
       [0,0,1,0]
       [0,0,1,0]
       [0,0,0,0]
       [0,0,1,0]

       Celebrity is person 2:
       - Everyone knows person 2 (column 2 all true)
```

```
       - Person 2 knows no one (row 2 all false)

       Two-pointer approach:
       candidate = 0
       For i = 1 to n-1:
           If knows[candidate][i] == true:
               candidate = i

       Verify candidate:
       For i = 0 to n-1:
           If i != candidate:
               If knows[candidate][i] == true: return -1
               If knows[i][candidate] == false: return -1
       Return candidate
```

Solution Approach

Concept: Elimination method with verification.

Algorithm

```
Algorithm CELEBRITY(matrix):
           n = matrix.length
           candidate = 0

            For i = 1 to n-1:
                If matrix[candidate][i]:
                    candidate = i

            For i = 0 to n-1:
                If i != candidate:
                    If matrix[candidate][i]: return -1
                    If !matrix[i][candidate]: return -1

            Return candidate
```

Java Implementation

```
class Solution {
           public int findCelebrity(int[][] matrix) {
               int n = matrix.length;
               int candidate = 0;

                 // Find candidate
                 for (int i = 1; i < n; i++) {
                     if (matrix[candidate][i]) {
                         candidate = i;

                    }
               }
```

```
            // Verify candidate
            for (int i = 0; i < n; i++) {
                if (i == candidate) continue;

                // Celebrity knows no one
                if (matrix[candidate][i]) {
                    return -1;
                }

                // Everyone knows celebrity
                if (!matrix[i][candidate]) {
                    return -1;
                }
            }
            return candidate;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(1)

197. LRU Cache

Understanding the Problem NEW

Design a cache with a fixed capacity that evicts the **Least Recently Used** item when it's full and a new item needs to be inserted — and supports both get and put in O(1) time.

Brute Force Approach NEW

A simple array or list-based cache could work, but finding and updating 'how recently used' each item is, or finding the least-recently-used item to evict, would require scanning the whole cache — O(n) per operation, failing the O(1) requirement.

Why We Optimize NEW

Combine a **hashmap** (for O(1) lookup of any key's location) with a **doubly linked list** (which naturally supports O(1) removal and reinsertion at any position, given a direct node reference, as established back in the LinkedList chapter). Keep the DLL ordered by recency, with the most-recently-used item near one end; on every access, move that item to the recency end in O(1) using the DLL's O(1) removal/reinsertion; when eviction is needed, remove the item at the opposite (least-recently-used) end. This combination gives true O(1) for both get and put, directly reusing the DLL's O(1) arbitrary-position deletion capability that was highlighted as a key DLL advantage back in the LinkedList chapter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement Least Recently Used cache.

Visual Diagram

```
Capacity = 3

      put(1,1): cache=[1]
      put(2,2): cache=[2,1]
      put(3,3): cache=[3,2,1]
      get(2): returns 2, cache=[2,3,1]
      put(4,4): cache full → remove 1, cache=[4,2,3]
```

```
get(1): returns -1
get(3): returns 3, cache=[3,4,2]

Data Structure:
HashMap for O(1) access
Doubly LinkedList for order
Head ←→ Most Recent ←→ Least Recent ←→ Tail
```

Solution Approach

Concept: HashMap + Doubly LinkedList.

Java Implementation

```
class LRUCache {
          private class Node {
              int key, value;
              Node prev, next;
              Node(int key, int value) {
                  this.key = key;
                  this.value = value;
              }
          }

          private int capacity;
          private Map<Integer, Node> map;
          private Node head, tail;

          public LRUCache(int capacity) {
              this.capacity = capacity;
              map = new HashMap<>();
              head = new Node(0, 0);
              tail = new Node(0, 0);
              head.next = tail;
              tail.prev = head;
          }

          public int get(int key) {
              if (!map.containsKey(key)) return -1;
              Node node = map.get(key);
              removeNode(node);
              addToFront(node);
              return node.value;
          }

          public void put(int key, int value) {
              if (map.containsKey(key)) {
                  Node node = map.get(key);

                   node.value = value;
                   removeNode(node);
```

```
                addToFront(node);
            } else {
                if (map.size() == capacity) {
                     Node lru = tail.prev;
                     removeNode(lru);
                     map.remove(lru.key);
                }
                Node newNode = new Node(key, value);
                map.put(key, newNode);
                addToFront(newNode);
            }
        }

        private void removeNode(Node node) {
            node.prev.next = node.next;
            node.next.prev = node.prev;
        }

        private void addToFront(Node node) {
            node.next = head.next;
            node.prev = head;
            head.next.prev = node;
            head.next = node;
        }
    }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(capacity)

198. LFU Cache

Understanding the Problem NEW

Similar to LRU Cache, but now eviction is based on **frequency of use** instead of recency — the Least Frequently Used item is evicted when the cache is full (with recency as a tiebreaker among equally-frequent items).

Brute Force Approach NEW

Track a frequency counter per key in a hashmap, and on eviction, scan all keys to find the minimum frequency (with some tiebreak logic) — O(n) per eviction, failing the O(1) goal.

Why We Optimize NEW

Maintain a hashmap from key to (value, frequency), *and* a second hashmap from frequency to a doubly linked list of all keys currently at that frequency (ordered by recency, so ties within the same frequency are broken LRU-style) — plus a running `minFrequency` counter. On every access, a key's frequency increases by 1, so it moves from its current frequency's DLL to the next frequency's DLL (each an O(1) DLL operation); eviction removes the least-recently-used key from the DLL at `minFrequency`. This layered structure — hashmap of DLLs, bucketed by frequency — achieves true O(1) for both get and put, extending LRU Cache's hashmap+DLL idea with an extra layer to also account for frequency.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Implement Least Frequently Used cache.

Visual Diagram

```
Capacity = 3

      put(1,1): freq=1 cache=[1]
      put(2,2): freq=1 cache=[1,2]
      get(1): freq=2 cache=[1,2]
      put(3,3): freq=1 cache=[3,1,2]
      get(2): freq=2 cache=[2,3,1]
      put(4,4): cache full → remove 3 (freq=1, LRU), cache=[4,2,3,1]
```

```
Data Structures:
HashMap: key → Node
HashMap: frequency → DoublyLinkedList
minFreq: track minimum frequency
```

Solution Approach

Concept: HashMap + Frequency Maps.

Java Implementation

```
class LFUCache {
          private class Node {
              int key, value, freq;
              Node prev, next;
              Node(int key, int value) {
                  this.key = key;
                  this.value = value;
                  this.freq = 1;
              }
          }

          private class DoublyLinkedList {
              Node head, tail;
              DoublyLinkedList() {
                  head = new Node(0, 0);
                  tail = new Node(0, 0);
                  head.next = tail;
                  tail.prev = head;
              }

               void addToFront(Node node) {
                   node.next = head.next;
                   node.prev = head;
                   head.next.prev = node;
                   head.next = node;
               }

               void removeNode(Node node) {
                   node.prev.next = node.next;
                   node.next.prev = node.prev;
               }

               Node removeLast() {
                   if (head.next == tail) return null;
                   Node last = tail.prev;
                   removeNode(last);

                   return last;
               }
```

```
        boolean isEmpty() {
            return head.next == tail;
        }
    }

    private int capacity, minFreq;
    private Map<Integer, Node> keyMap;
    private Map<Integer, DoublyLinkedList> freqMap;

    public LFUCache(int capacity) {
        this.capacity = capacity;
        minFreq = 0;
        keyMap = new HashMap<>();
        freqMap = new HashMap<>();
    }

    public int get(int key) {
        if (!keyMap.containsKey(key)) return -1;
        Node node = keyMap.get(key);
        updateFreq(node);
        return node.value;
    }

    public void put(int key, int value) {
        if (capacity == 0) return;

        if (keyMap.containsKey(key)) {
            Node node = keyMap.get(key);
            node.value = value;
            updateFreq(node);
            return;
        }

        if (keyMap.size() == capacity) {
            DoublyLinkedList list = freqMap.get(minFreq);
            Node lru = list.removeLast();
            keyMap.remove(lru.key);
        }

        Node newNode = new Node(key, value);
        keyMap.put(key, newNode);
        minFreq = 1;
        freqMap.computeIfAbsent(1, k -> new DoublyLinkedList())
               .addToFront(newNode);
    }

    private void updateFreq(Node node) {
        int freq = node.freq;
        DoublyLinkedList list = freqMap.get(freq);
        list.removeNode(node);

        if (freq == minFreq && list.isEmpty()) {
            minFreq++;
```

```
            }

            node.freq++;
            freqMap.computeIfAbsent(node.freq, k -> new DoublyLinkedList())
                    .addToFront(node);
        }
    }
```

Time & Space Complexity

Time Complexity: O(1) | Space Complexity: O(capacity)

Sliding Window & Two Pointer — Complete Guide

199. Longest Substring Without Repeating Characters

Understanding the Problem NEW

Given a string, find the length of the longest substring that doesn't contain any repeated characters. E.g. `"abcabcbb"` → the longest such substring is `"abc"`, length 3.

Brute Force Approach NEW

Check every possible substring and verify it has no repeated characters — $O(n^2)$ or $O(n^3)$ depending on how the repeat-check is done per substring.

Why We Optimize NEW

Use a **sliding window** with a hashmap tracking the last seen index of each character: expand the window's right edge one character at a time; if the new character has already been seen *within the current window*, jump the window's left edge forward to just past that character's last occurrence (rather than shrinking one step at a time), keeping the window always free of duplicates. Because both the left and right edges only ever move forward, never backward, the whole scan is O(n) — this exact sliding-window shape (expand right, jump/shrink left when a constraint is violated) is the template for essentially every remaining problem in this chapter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the longest substring without repeating characters.

Visual Diagram

```
Input: "abcabcbb"
       Output: 3 ("abc" or "bca" or "cab")

       Sliding Window:
       i=0, j=0: map={}, max=0
       a: map={a:1}, max=1, j=1
       b: map={a:1,b:1}, max=2, j=2
       c: map={a:1,b:1,c:1}, max=3, j=3
```

```
       a: map contains a → move i to index of a+1 (1)
          map={b:1,c:1,a:1}, max=3, j=4
       b: map contains b → move i to index of b+1 (2)
          map={c:1,a:1,b:1}, max=3, j=5
       c: map contains c → move i to index of c+1 (3)
          map={a:1,b:1,c:1}, max=3, j=6
       b: map contains b → move i to index of b+1 (4)
          max=3
       Result: 3

       Input: "bbbbb"
       Output: 1

       i=0, j=0: b → max=1
       j=1: b → i moves to 1
       j=2: b → i moves to 2
       ...
       max=1
```

Solution Approach

Concept: Sliding window with hashmap tracking last occurrence.

Algorithm

```
Algorithm LONGEST_UNIQUE_SUBSTRING(s):
           map = new HashMap<Character, Integer>()
           left = 0, maxLen = 0

           For right = 0 to n-1:
               If map contains s[right]:
                   left = max(left, map.get(s[right]) + 1)
               map.put(s[right], right)
               maxLen = max(maxLen, right - left + 1)

           Return maxLen
```

Java Implementation

```
class Solution {
           public int lengthOfLongestSubstring(String s) {
               Map<Character, Integer> map = new HashMap<>();
               int left = 0, maxLen = 0;

               for (int right = 0; right < s.length(); right++) {
                   char c = s.charAt(right);
                   if (map.containsKey(c)) {
                       left = Math.max(left, map.get(c) + 1);
                   }
                   map.put(c, right);
```

```
                maxLen = Math.max(maxLen, right - left + 1);
            }
            return maxLen;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(min(n, 26))

200. Max Consecutive Ones III

Understanding the Problem NEW

Given a binary array, find the longest subarray of all 1s achievable if you're allowed to flip at most K zeros to 1s.

Brute Force Approach NEW

Try every possible subarray, count its zeros, and check if that count is ≤ K — $O(n^2)$.

Why We Optimize NEW

Sliding window: expand the window's right edge, tracking a running count of zeros inside it; whenever that zero count exceeds K, shrink from the left until it's back within the allowed K — tracking the maximum window size seen throughout. Same 'expand right, shrink left on violation' template as problem 199, just with a numeric zero-count constraint.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find max consecutive ones with at most k zeros flipped.

Visual Diagram

```
Input: [1,1,1,0,0,0,1,1,1,1,0], k=2
       Output: 6

       Sliding Window:
       Window [1,1,1,0,0] → zeros=2, len=5
       Window [1,1,1,0,0,0] → zeros=3 > k → shrink
       Window [1,1,0,0,0] → zeros=3 > k → shrink
       Window [1,0,0,0] → zeros=3 > k → shrink
       Window [0,0,0] → zeros=3 > k → shrink
       Window [0,0,0,1,1,1,1] → zeros=3 > k → shrink
       Window [0,0,1,1,1,1,0] → zeros=3 > k → shrink
       Window [0,1,1,1,1,0] → zeros=2, len=6
       Result: 6

       Sliding Window:
       left=0, zeros=0, maxLen=0
```

```
right=0: 1 → zeros=0, len=1
right=1: 1 → zeros=0, len=2
right=2: 1 → zeros=0, len=3
right=3: 0 → zeros=1, len=4
right=4: 0 → zeros=2, len=5
right=5: 0 → zeros=3 → shrink
left=1: zeros=2 (remove first 1)
len=5
right=6: 1 → zeros=2, len=6
...
maxLen=6
```

Solution Approach

Concept: Sliding window tracking zeros count.

Algorithm

```
Algorithm MAX_CONSECUTIVE_ONES(nums, k):
           left = 0, zeros = 0, maxLen = 0

           For right = 0 to n-1:
               If nums[right] == 0:
                   zeros++

                 While zeros > k:
                     If nums[left] == 0:
                         zeros--
                     left++

                 maxLen = max(maxLen, right - left + 1)

           Return maxLen
```

Java Implementation

```
class Solution {
           public int longestOnes(int[] nums, int k) {
               int left = 0, zeros = 0, maxLen = 0;

              for (int right = 0; right < nums.length; right++) {
                  if (nums[right] == 0) {
                      zeros++;
                  }

                  while (zeros > k) {
                      if (nums[left] == 0) {
                          zeros--;
                      }
                      left++;
```

```
                }

                maxLen = Math.max(maxLen, right - left + 1);
            }
            return maxLen;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

201. Fruit Into Baskets

Understanding the Problem NEW

Given an array representing fruit types on trees in a row, find the longest subarray (contiguous run of trees) that contains at most 2 distinct fruit types — a real-world framing of 'longest subarray with at most K distinct elements' where K=2.

Brute Force Approach NEW

Check every subarray and count its distinct fruit types — $O(n^2)$ or $O(n^3)$.

Why We Optimize NEW

Sliding window with a hashmap tracking counts of each fruit type currently in the window: expand right, and whenever the window contains more than 2 distinct types, shrink from the left (decrementing/removing counts) until back down to 2 or fewer types — tracking the maximum window length. Exactly the same template again, this time with 'number of distinct keys in the frequency map' as the constraint being checked.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find longest subarray with at most 2 distinct elements.

Visual Diagram

```
Input: [1,2,1,2,3,2,3,1,1]
       Output: 4

       Sliding Window:
       Window [1,2,1,2] → 2 types, len=4
       When 3 appears: [1,2,1,2,3] → 3 types → shrink from left
       After shrink: [2,3,2,3,1,1] → Wait, let's trace properly

       Left=0, right=0: {1:1} → max=1
       right=1: {1:1,2:1} → max=2
       right=2: {1:2,2:1} → max=3
       right=3: {1:2,2:2} → max=4
       right=4: {1:2,2:2,3:1} → 3 types → shrink
```

```
          left=1: {1:1,2:2,3:1} → still 3
          left=2: {1:0,2:2,3:1} → 2 types, len=3
       right=5: {2:2,3:2} → len=4
       right=6: {2:2,3:2,1:1} → 3 types → shrink
          left=3: {2:1,3:2,1:1} → still 3
          left=4: {2:0,3:2,1:1} → 2 types, len=3
       right=7: {3:2,1:2} → len=4
       right=8: {3:2,1:3} → len=5? Wait, window [3,1,1] length 3
       Actually max=4

       Result: 4
```

Solution Approach

Concept: Sliding window with HashMap for distinct elements.

Algorithm

```
Algorithm FRUIT_INTO_BASKETS(fruits):
           left = 0, maxLen = 0
           map = new HashMap()

           For right = 0 to n-1:
               map.put(fruits[right], map.getOrDefault(fruits[right], 0) + 1)

                 While map.size() > 2:
                     map.put(fruits[left], map.get(fruits[left]) - 1)
                     If map.get(fruits[left]) == 0:
                         map.remove(fruits[left])
                     left++

                 maxLen = Math.max(maxLen, right - left + 1)

           Return maxLen
```

Java Implementation

```
class Solution {
           public int totalFruit(int[] fruits) {
               Map<Integer, Integer> map = new HashMap<>();
               int left = 0, maxLen = 0;
               for (int right = 0; right < fruits.length; right++) {

                  map.put(fruits[right], map.getOrDefault(fruits[right], 0) + 1);

                  while (map.size() > 2) {
                      map.put(fruits[left], map.get(fruits[left]) - 1);
                      if (map.get(fruits[left]) == 0) {
                          map.remove(fruits[left]);
                      }
```

```
                    left++;
                }

                maxLen = Math.max(maxLen, right - left + 1);
            }
            return maxLen;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

202. Longest Repeating Character Replacement

Understanding the Problem NEW

Given a string, find the length of the longest substring you can make entirely one repeated character, if you're allowed to change at most K characters.

Brute Force Approach NEW

Check every substring, count its most frequent character, and verify (length - max frequency) ≤ K — O(n²) or worse.

Why We Optimize NEW

Sliding window with a character frequency map: expand right, tracking the count of the *most frequent* character seen so far in the current window; the window is valid as long as `(window length - most frequent count) ≤ K` (meaning you'd need to change at most K characters to make the window uniform); if this is violated, shrink from the left. A subtle but important optimization: you don't need to recompute the true most-frequent-count exactly every time you shrink — you can allow it to be a slightly stale (but never overestimated) value and still get the correct final answer, since the window only grows when a strictly better max frequency is found. Same expand-right/shrink-left template, adapted with this frequency-based constraint.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Replace at most k characters to make longest repeating substring.

Visual Diagram

```
Input: "ABAB", k=2
       Output: 4 ("ABAB" → "AAAA")

       Window approach:
       right=0: 'A' → maxFreq=1, len=1, changes=0
       right=1: 'B' → maxFreq=1, len=2, changes=1
       right=2: 'A' → maxFreq=2, len=3, changes=1
       right=3: 'B' → maxFreq=2, len=4, changes=2
       maxLen=4
```

```
Input: "AABABBA", k=1
Output: 4

Window:
right=0: A → maxFreq=1, len=1
right=1: A → maxFreq=2, len=2
right=2: B → maxFreq=2, len=3, changes=1
right=3: A → maxFreq=3, len=4, changes=1
right=4: B → maxFreq=3, len=5, changes=2 > k → shrink
left=1: remove A → maxFreq still 3? No, recalc maxFreq
Actually maxFreq should be maintained differently

Better approach: maxFreq is max frequency in window
```

Solution Approach

Concept: Sliding window with character frequency.

Algorithm

```
Algorithm CHARACTER_REPLACEMENT(s, k):
           left = 0, maxFreq = 0, maxLen = 0
           freq = new int[26]

           For right = 0 to n-1:
               freq[s[right]-'A']++
               maxFreq = max(maxFreq, freq[s[right]-'A'])

                 // If window length - maxFreq > k, shrink
                 While (right - left + 1) - maxFreq > k:
                     freq[s[left]-'A']--
                     left++
                     // Recalculate maxFreq
                     maxFreq = getMaxFreq(freq)

                 maxLen = max(maxLen, right - left + 1)

           Return maxLen
```

Java Implementation

```
class Solution {
           public int characterReplacement(String s, int k) {
               int[] freq = new int[26];
               int left = 0, maxFreq = 0, maxLen = 0;

              for (int right = 0; right < s.length(); right++) {
                  freq[s.charAt(right) - 'A']++;
                  maxFreq = Math.max(maxFreq, freq[s.charAt(right) - 'A']);
```

```
                int windowLen = right - left + 1;
                if (windowLen - maxFreq > k) {
                    freq[s.charAt(left) - 'A']--;
                    left++;
                    // Recalculate maxFreq
                    maxFreq = getMaxFreq(freq);
                }

                maxLen = Math.max(maxLen, right - left + 1);
            }
            return maxLen;
        }

        private int getMaxFreq(int[] freq) {
            int max = 0;
            for (int f : freq) {
                max = Math.max(max, f);
            }
            return max;
        }
    }
```

Time & Space Complexity

Time Complexity: O(26*n) = O(n) | Space Complexity: O(1)

203. Binary Subarray with Sum

Understanding the Problem NEW

Given a binary array and a target sum, count how many subarrays sum to exactly that target.

Brute Force Approach NEW

Check every subarray's sum directly — O(n²).

Why We Optimize NEW

Since all elements are 0 or 1, you could use the prefix-sum-plus-hashmap technique from the Arrays chapter (problem 41) directly — but there's also a neat sliding-window alternative specific to non-negative integers: compute `atMost(target) - atMost(target-1)`, where `atMost(k)` counts subarrays with sum ≤ k using a simple sliding window (since sums only grow as the window expands, thanks to non-negative values). This 'exactly K = atMost(K) - atMost(K-1)' trick is a reusable pattern for turning an 'at most' sliding-window count into an 'exactly' count, and reappears in problems 204 and 208.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count subarrays with sum equal to target in binary array.

Visual Diagram

```
Input: [1,0,1,0,1], target=2
       Output: 4

       Subarrays with sum 2:
       [1,0,1] (0-2)
       [1,0,1,0] (0-3)
       [0,1,0,1] (1-4)
       [1,0,1] (2-4)
       Total: 4

       Two methods:
       1. Sliding Window (for non-negative)
       2. Prefix Sum with HashMap
```

```
Method 1 (Sliding Window):
For each left, find right for target and target-1
Count = atMost(target) - atMost(target-1)

Method 2 (Prefix Sum):
prefix = 0, map = {0:1}
For each num:
    prefix += num
    count += map.getOrDefault(prefix - target, 0)
    map.put(prefix, map.getOrDefault(prefix, 0) + 1)
```

Solution Approach

Concept: Prefix sum or sliding window.

Algorithm (Prefix Sum)

Algorithm BINARY_SUBARRAY_SUM(nums, target):

map = {0: 1}

prefix = 0, count = 0

For num in nums:

prefix += num

count += map.getOrDefault(prefix - target, 0)

map.put(prefix, map.getOrDefault(prefix, 0) + 1)

Return count

Java Implementation (Prefix Sum)

class Solution {

public int numSubarraysWithSum(int[] nums, int goal) {

Map<Integer, Integer> map = new HashMap<>();

map.put(0, 1);

int prefix = 0, count = 0;

for (int num : nums) {

prefix += num;

count += map.getOrDefault(prefix - goal, 0);

```
map.put(prefix, map.getOrDefault(prefix, 0) + 1);
}
return count;
}
}
```

Java Implementation (Sliding Window)

```
class Solution {
public int numSubarraysWithSum(int[] nums, int goal) {
return atMost(nums, goal) - atMost(nums, goal - 1);
}
private int atMost(int[] nums, int goal) {
if (goal < 0) return 0;
int left = 0, sum = 0, count = 0;
for (int right = 0; right < nums.length; right++) {
sum += nums[right];
while (sum > goal) {
sum -= nums[left];
left++;
}
count += right - left + 1;
}
return count;
}
}
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n) or O(1)

204. Count Number of Nice Subarrays

Understanding the Problem NEW

Count subarrays containing exactly K odd numbers.

Brute Force Approach NEW

Check every subarray and count its odd numbers directly — $O(n^2)$.

Why We Optimize NEW

Transform the problem: replace every even number with 0 and every odd number with 1, turning this into exactly the 'Binary Subarray with Sum' problem (203) with target K — and reuse the same `atMost(K) - atMost(K-1)` sliding-window trick. Recognizing that this 'different-looking' problem is actually a disguised version of one you already solved (by a simple even/odd → 0/1 relabeling) is the real insight here.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count subarrays with exactly k odd numbers.

Visual Diagram

```
Input: [1,1,2,1,1], k=3
       Output: 2

       Subarrays with exactly 3 odd numbers:
       [1,1,2,1] (0-3) → 3 odds
       [1,2,1,1] (1-4) → 3 odds
       Total: 2

       Method: Same as binary subarray with sum
       Treat odd as 1, even as 0
       Find subarrays with sum = k

       Using sliding window:
       count = atMost(k) - atMost(k-1)
```

Solution Approach

Concept: Convert to binary array.

Algorithm

```
Algorithm NICE_SUBARRAYS(nums, k):
           For i = 0 to n-1:
               nums[i] = nums[i] % 2

           Return atMost(nums, k) - atMost(nums, k-1)
```

Java Implementation

```
class Solution {
           public int numberOfSubarrays(int[] nums, int k) {
               for (int i = 0; i < nums.length; i++) {
                   nums[i] = nums[i] % 2;
               }
               return atMost(nums, k) - atMost(nums, k - 1);
           }

           private int atMost(int[] nums, int k) {
               if (k < 0) return 0;
               int left = 0, sum = 0, count = 0;

                 for (int right = 0; right < nums.length; right++) {
                     sum += nums[right];
                     while (sum > k) {
                         sum -= nums[left];
                         left++;
                     }
                     count += right - left + 1;
                 }
                 return count;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

205. Number of Substrings Containing All Three Characters

Understanding the Problem NEW

Given a string of only 'a', 'b', and 'c' characters, count the number of substrings that contain *at least one* of each of the three characters.

Brute Force Approach NEW

Check every substring and verify it contains all three characters — $O(n^2)$ or $O(n^3)$.

Why We Optimize NEW

Sliding window tracking the last-seen index of 'a', 'b', and 'c': for each right edge position, once the window contains all three characters, *every* substring starting anywhere from the current left edge up to the earliest of the three last-seen positions, and ending at the current right edge, must also contain all three — so you can add `(1 + min(last-seen positions) - left)` to the count in O(1) per right-edge step, rather than checking each qualifying substring individually. This is a slightly different flavor of sliding window — counting all valid substrings ending at each position in bulk, rather than tracking a single best window length — but built on the same expand-right-and-reason-about-what's-now-valid idea.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count substrings containing 'a', 'b', and 'c'.

Visual Diagram

```
Input: "abcabc"
        Output: 10

        Substrings containing a,b,c:
        All substrings with length >=3:
        "abc" (0-2)
        "abca" (0-3)
        "abcab" (0-4)
        "abcabc" (0-5)
        "bca" (1-3)
```

```
"bcab" (1-4)
"bcabc" (1-5)
"cab" (2-4)
"cabc" (2-5)
"abc" (3-5)
Total: 10

Sliding Window:
left=0, right=0: "a" → no
right=1: "ab" → no
right=2: "abc" → yes, count += n-right = 6-2=4
   left=1 → "bc" → no
right=3: "bca" → yes, count += 6-3=3
   left=2 → "ca" → no
right=4: "cab" → yes, count += 6-4=2
   left=3 → "ab" → no
right=5: "abc" → yes, count += 6-5=1
Total: 4+3+2+1=10
```

Solution Approach

Concept: Sliding window with frequency array.

Algorithm

```
Algorithm SUBSTRINGS_ALL_THREE(s):
            left = 0, count = 0
            freq = new int[3]

            For right = 0 to n-1:
                freq[s[right]-'a']++

                 While freq[0] > 0 && freq[1] > 0 && freq[2] > 0:
                     count += n - right
                     freq[s[left]-'a']--
                     left++

            Return count
```

Java Implementation

```
class Solution {
            public int numberOfSubstrings(String s) {
                int[] freq = new int[3];
                int left = 0, count = 0;

              int n = s.length();

              for (int right = 0; right < n; right++) {
                  freq[s.charAt(right) - 'a']++;
```

```
                while (freq[0] > 0 && freq[1] > 0 && freq[2] > 0) {
                    count += n - right;
                    freq[s.charAt(left) - 'a']--;
                    left++;
                }
            }
            return count;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

206. Maximum Points You Can Obtain from Cards

Understanding the Problem NEW

Given a row of cards each with a point value, and you must pick exactly K cards total, taking each pick from either the very front or the very back of the row — maximize total points.

Brute Force Approach NEW

Try every possible way of splitting K picks between 'taken from front' and 'taken from back' by simulating each split explicitly and summing — this is actually feasible in O(K) directly, but a truly naive recursive/exhaustive exploration of every pick order would be exponential.

Why We Optimize NEW

Reframe the problem: picking K cards from the two ends is equivalent to **leaving behind** a single contiguous subarray of length `n-K` from the middle (whatever you don't pick must be one unbroken chunk in the middle, since all picks come from the two ends). So the problem becomes: find the **minimum sum** contiguous subarray of length exactly `n-K`, and subtract it from the total sum of all cards — solvable with a simple fixed-size sliding window in O(n), turning an 'exploration of front/back split combinations' framing into a much simpler 'minimum fixed-window sum' framing.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Maximum points by taking cards from either end.

Visual Diagram

```
Input: [1,2,3,4,5,6,1], k=3
       Output: 12

       Take from ends:
       Left: 1+2+3=6
       Right: 1+6+5=12
       Left: 1+6+1=8
       Right: 6+5+4=15? Wait k=3, can only take 3
```

```
Maximum: 12 (take 1,6,5 from right)

Sliding Window:
Take all k from left: [1,2,3] sum=6
Then slide:
Remove left, add right: [2,3,4? no, rightmost 1]
Actually: total = sum of all elements - sum of middle window of size n-k

n=7, k=3, window size = 4
Find minimum sum of 4 consecutive elements
Total sum = 22
Min window sum = 2+3+4+5=14
Max points = 22-14=8? Wait that's wrong

Actually:
k=3, can take from ends
Window of size n-k = 4 will be the remaining cards
We want min sum of 4 consecutive cards
Total sum = 22, min window sum = 2+3+4+5=14
Max points = 22-14=8

But expected is 12... Let's try:
Take all 3 from right: [1,6,5] = 12

The sliding window approach:
Take k from left: [1,2,3] = 6
Then slide window by removing from left and adding from right:
[1,2,3] → remove 1, add 1 → [2,3,1] = 6
[2,3,1] → remove 2, add 6 → [3,1,6] = 10
[3,1,6] → remove 3, add 5 → [1,6,5] = 12
Max = 12
```

Solution Approach

Concept: Sliding window on circular array.

Algorithm

```
Algorithm MAX_POINTS(cardPoints, k):
          n = cardPoints.length
          total = sum(cardPoints)
          windowSize = n - k
          minSum = sum of first windowSize elements
          currentSum = minSum

          For i = windowSize to n-1:

             currentSum = currentSum - cardPoints[i-windowSize] + cardPoints[i]
             minSum = min(minSum, currentSum)
```

```
          Return total - minSum
```

Java Implementation

```
class Solution {
          public int maxScore(int[] cardPoints, int k) {
              int n = cardPoints.length;
              int total = 0;
              for (int points : cardPoints) {
                  total += points;
              }

              int windowSize = n - k;
              int currentSum = 0;
              for (int i = 0; i < windowSize; i++) {
                  currentSum += cardPoints[i];
              }

              int minSum = currentSum;
              for (int i = windowSize; i < n; i++) {
                  currentSum = currentSum - cardPoints[i - windowSize] +
cardPoints[i];
                  minSum = Math.min(minSum, currentSum);
              }

              return total - minSum;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

207. Longest Substring with At Most K Distinct Characters

Understanding the Problem NEW

Find the longest substring containing at most K distinct characters — the general form of the Fruit Into Baskets problem (201), which was this same problem with K fixed at 2.

Brute Force Approach NEW

Check every substring and count its distinct characters — $O(n^2)$ or $O(n^3)$.

Why We Optimize NEW

Exactly the same sliding window with a frequency hashmap as problem 201, just with a general K instead of hardcoded 2: expand right, and if the number of distinct characters (keys in the frequency map) exceeds K, shrink from the left until it's back within K, tracking the maximum window length. This problem exists specifically to make explicit that Fruit Into Baskets was really 'K=2' of this more general template all along.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find longest substring with at most k distinct characters.

Visual Diagram

```
Input: "eceba", k=2
       Output: 3 ("ece")

       Sliding Window:
       left=0, right=0: "e" → {e:1}, len=1
       right=1: "ec" → {e:1,c:1}, len=2
       right=2: "ece" → {e:2,c:1}, len=3
       right=3: "eceb" → {e:2,c:1,b:1} → 3 distinct > k
          left=1: "ceb" → {e:1,c:1,b:1} → still 3
          left=2: "eb" → {e:1,b:1} → 2 distinct, len=2
       right=4: "eba" → {e:1,b:1,a:1} → 3 distinct
          left=3: "ba" → {b:1,a:1} → len=2
       maxLen=3
```

```
Input: "aa", k=1
Output: 2
```

Solution Approach

Concept: Sliding window with frequency map.

Algorithm

```
Algorithm LONGEST_K_DISTINCT(s, k):
           left = 0, maxLen = 0
           map = new HashMap()

           For right = 0 to n-1:
               map.put(s[right], map.getOrDefault(s[right], 0) + 1)

                 While map.size() > k:
                     map.put(s[left], map.get(s[left]) - 1)
                     If map.get(s[left]) == 0:
                         map.remove(s[left])
                     left++

                 maxLen = max(maxLen, right - left + 1)

           Return maxLen
```

Java Implementation

```
class Solution {
           public int lengthOfLongestSubstringKDistinct(String s, int k) {
               if (k == 0 || s.isEmpty()) return 0;

                 Map<Character, Integer> map = new HashMap<>();
                 int left = 0, maxLen = 0;

                 for (int right = 0; right < s.length(); right++) {
                     map.put(s.charAt(right), map.getOrDefault(s.charAt(right), 0)
+ 1);
                     while (map.size() > k) {

                      char leftChar = s.charAt(left);
                      map.put(leftChar, map.get(leftChar) - 1);
                      if (map.get(leftChar) == 0) {
                          map.remove(leftChar);
                      }
                      left++;
                  }

                  maxLen = Math.max(maxLen, right - left + 1);
```

```
            }
            return maxLen;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(k)

208. Subarray with K Different Integers

Understanding the Problem NEW

Count subarrays with **exactly** K distinct integers — the 'exactly' version building on the 'at most K distinct' sliding window from problem 207.

Brute Force Approach NEW

Check every subarray and count its distinct values — $O(n^2)$ or $O(n^3)$.

Why We Optimize NEW

Reuse the same `atMost(K) - atMost(K-1)` transformation from problems 203/204: build an `atMost(K distinct)` helper using the sliding window from problem 207 (which counts subarrays with at most K distinct values, by adding `(right - left + 1)` new valid subarrays at each right-edge step), then the answer is `atMost(K) - atMost(K-1)`. This is now the third appearance of the 'exactly = atMost(K) - atMost(K-1)' pattern in this chapter, worth committing to memory as a general technique whenever a sliding window naturally handles 'at most' constraints more easily than 'exactly' ones.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count subarrays with exactly k distinct integers.

Visual Diagram

```
Input: [1,2,1,2,3], k=2
       Output: 7

       Subarrays with exactly 2 distinct:
       [1,2] (0-1)
       [1,2,1] (0-2)
       [1,2,1,2] (0-3)
       [2,1] (1-2)
       [2,1,2] (1-3)
       [1,2] (2-3)
       [2,3] (3-4)
       Total: 7
```

```
       Method: atMost(k) - atMost(k-1)
```

Solution Approach

Concept: Sliding window with atMost function.

Algorithm

```
Algorithm SUBARRAYS_K_DISTINCT(nums, k):
           Return atMost(nums, k) - atMost(nums, k-1)

       Algorithm atMost(nums, k):
           left = 0, count = 0
           map = new HashMap()

           For right = 0 to n-1:
               map.put(nums[right], map.getOrDefault(nums[right], 0) + 1)

                 While map.size() > k:
                     map.put(nums[left], map.get(nums[left]) - 1)
                     If map.get(nums[left]) == 0:
                         map.remove(nums[left])
                     left++

                 count += right - left + 1

           Return count
```

Java Implementation

```
class Solution {
           public int subarraysWithKDistinct(int[] nums, int k) {
               return atMost(nums, k) - atMost(nums, k - 1);
           }

           private int atMost(int[] nums, int k) {
               if (k < 0) return 0;

                 Map<Integer, Integer> map = new HashMap<>();
                 int left = 0, count = 0;

                 for (int right = 0; right < nums.length; right++) {

                  map.put(nums[right], map.getOrDefault(nums[right], 0) + 1);

                  while (map.size() > k) {
                      map.put(nums[left], map.get(nums[left]) - 1);
                      if (map.get(nums[left]) == 0) {
                          map.remove(nums[left]);
```

```
                    }
                    left++;
                }

                count += right - left + 1;
            }
            return count;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(k)

209. Minimum Window Substring

Understanding the Problem NEW

Given a string S and a target string T, find the smallest substring (window) of S that contains every character of T (including duplicates — if T has two 'a's, the window needs at least two 'a's too).

Brute Force Approach NEW

Check every substring of S and verify it contains all required characters (with correct counts) — $O(n^2)$ substrings, each requiring an O(n) or O(alphabet) check, giving $O(n^2)$ to $O(n^3)$ total.

Why We Optimize NEW

Sliding window with a frequency map of T's required characters and a counter of 'how many required character-slots are currently satisfied': expand the right edge, and whenever the window fully satisfies every requirement, try **shrinking from the left** as much as possible while still satisfying every requirement, recording the smallest valid window found along the way. Because both edges only move forward overall (even though the left edge does its own shrink loop), the total work across the whole scan is O(n + m) — a more intricate version of the same expand-right/shrink-left template, adapted to track multiple required-character counts simultaneously instead of a single numeric constraint.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find minimum window containing all characters of target.

Visual Diagram

```
Input: s = "ADOBECODEBANC", t = "ABC"
       Output: "BANC"

       Step-by-step:
       Window [A D O B E C] (0-5) → contains A,B,C, len=6
       Window [D O B E C O D E B A N C] → wait, let's trace

       left=0, right=0: "A" → missing B,C
```

```
right=1: "AD" → missing B,C
right=2: "ADO" → missing B,C
right=3: "ADOB" → missing C
right=4: "ADOBE" → missing C
right=5: "ADOBEC" → found! len=6, min="ADOBEC"
Shrink left:
left=1: "DOBEC" → missing A
left=2: "OBEC" → missing A
...
right=6: "DOBECOD" → missing A
right=7: "DOBECODE" → missing A
right=8: "DOBECODEB" → missing A
right=9: "DOBECODEBA" → found! len=10
Shrink left...
When right=12: "BANC" → found! len=4
Result: "BANC"
```

Solution Approach

Concept: Sliding window with character count.

Algorithm

```
Algorithm MIN_WINDOW(s, t):
           If s.length() < t.length(): Return ""

           map = new HashMap()
           For char in t: map.put(char, map.getOrDefault(char,0)+1)

           left = 0, minLen = INF, minStart = 0
           count = t.length() // characters to match

           For right = 0 to n-1:
               char c = s[right]
               If map.containsKey(c):
                   If map.get(c) > 0: count--
                   map.put(c, map.get(c)-1)

                 While count == 0:
                     If right - left + 1 < minLen:
                         minLen = right - left + 1
                         minStart = left

                    char leftChar = s[left]
                    If map.containsKey(leftChar):
                        map.put(leftChar, map.get(leftChar)+1)
                        If map.get(leftChar) > 0: count++

                  left++
```

```
        Return minLen == INF ? "" : s.substring(minStart, minStart+minLen)
```

Java Implementation

```
class Solution {
          public String minWindow(String s, String t) {
              if (s.length() < t.length()) return "";

              Map<Character, Integer> map = new HashMap<>();
              for (char c : t.toCharArray()) {
                  map.put(c, map.getOrDefault(c, 0) + 1);
              }

              int left = 0, minLen = Integer.MAX_VALUE, minStart = 0;
              int count = t.length();

              for (int right = 0; right < s.length(); right++) {
                  char c = s.charAt(right);
                  if (map.containsKey(c)) {
                      if (map.get(c) > 0) count--;
                      map.put(c, map.get(c) - 1);
                  }

                  while (count == 0) {
                      int windowLen = right - left + 1;
                      if (windowLen < minLen) {
                          minLen = windowLen;
                          minStart = left;
                      }

                      char leftChar = s.charAt(left);
                      if (map.containsKey(leftChar)) {
                          map.put(leftChar, map.get(leftChar) + 1);
                          if (map.get(leftChar) > 0) count++;
                      }
                      left++;
                  }
              }

              return minLen == Integer.MAX_VALUE ? "" : s.substring(minStart,
minStart + minLen);
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(m)

210. Minimum Window Subsequence

Understanding the Problem NEW

A variant of Minimum Window Substring: find the minimum window in S that contains T as a **subsequence** (characters of T must appear in the same relative order within the window, but don't need to be contiguous — unlike problem 209, where only the character counts mattered, not their order).

Brute Force Approach NEW

Check every substring of S and verify T is a valid subsequence of it (an O(n) check per substring using two pointers) — $O(n^2)$ to $O(n^3)$ total depending on substring generation cost.

Why We Optimize NEW

Because *order* matters here (unlike problem 209's frequency-only requirement), a frequency-map sliding window isn't sufficient — instead, use a **two-pointer subsequence-matching scan**: walk forward through S matching characters of T in order until a full match is found (marking a candidate window's end), then walk *backward* from that point re-matching T in reverse to find the tightest possible start of that window, then continue scanning forward in S from just after the previous match's start to look for the next candidate window. This differs from the standard sliding window template specifically because subsequence matching (not just character-count satisfaction) requires tracking positional order, not just counts — a good example of recognizing when the general sliding-window template needs a genuinely different mechanism rather than just a different constraint check.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find minimum window in s containing t as subsequence.

Visual Diagram

```
Input: s = "abcdebdde", t = "bde"
      Output: "bcde"

      Step-by-step:
```

```
Find t as subsequence:
s[1]='b', s[3]='d', s[4]='e' → "bcde" len=4
s[1]='b', s[3]='d', s[7]='e' → "bcdde" len=5
s[1]='b', s[6]='d', s[7]='e' → "bcde" len=4
s[5]='b', s[6]='d', s[7]='e' → "bde" len=3
Minimum: "bcde" (4)

Two-pointer approach:
For each possible start:
  Find end where subsequence ends
  Then find start of subsequence
  Update minimum

Using DP for subsequence matching:
dp[i][j] = start position of subsequence
```

Solution Approach

Concept: Two-pointer subsequence matching.

Algorithm

```
Algorithm MIN_WINDOW_SUBSEQUENCE(s, t):
          n = s.length(), m = t.length()
          minLen = INF, minStart = -1

          For i = 0 to n-1:
              // Find end where subsequence matches
              j = 0
              While i < n && j < m:
                  If s[i] == t[j]: j++
                  i++

                 If j == m:
                     // Found subsequence ending at i-1
                     end = i-1
                     // Find start
                     j = m-1
                     start = end
                     While j >= 0:
                         If s[start] == t[j]: j--
                         start--
                     start++

                    If end - start + 1 < minLen:
                        minLen = end - start + 1
                        minStart = start

                    i = start + 1 // Reset for next search
```

```
        Return minLen == INF ? "" : s.substring(minStart, minStart+minLen)
```

Java Implementation

```
class Solution {
           public String minWindow(String s, String t) {
               int n = s.length(), m = t.length();
               int minLen = Integer.MAX_VALUE;
               int minStart = -1;

               for (int i = 0; i < n; i++) {
                   // Find end where t is subsequence
                   int j = 0;
                   int temp = i;
                   while (temp < n && j < m) {
                       if (s.charAt(temp) == t.charAt(j)) {
                           j++;
                       }
                       temp++;
                   }

                   if (j == m) {
                       // Found subsequence, now find the start
                       int end = temp - 1;
                       j = m - 1;
                       int start = end;
                       while (j >= 0) {
                           if (s.charAt(start) == t.charAt(j)) {
                                j--;
                           }
                           start--;
                       }
                       start++;

                       if (end - start + 1 < minLen) {
                           minLen = end - start + 1;
                           minStart = start;
                       }

                       // Reset to search for next window
                       i = start;
                   }
               }

               return minStart == -1 ? "" : s.substring(minStart, minStart +
minLen);
           }
       }
```

Time & Space Complexity

Time Complexity: O(n^2) worst case | Space Complexity: O(1)

Stack & Queue Summary

Problem	Type	Time	Space	Key Technique
Stack Array	Basic	O(1)	O(n)	Array with top
Queue Array	Basic	O(1)	O(n)	Circular array
Stack using Queue	Basic	Push O(n), Pop O(1)	O(n)	Two queues
Queue using Stack	Basic	Push O(1), Pop O(n)	O(n)	Two stacks
Stack LL	Basic	O(1)	O(n)	Linked list
Queue LL	Basic	O(1)	O(n)	Linked list
Balanced Parentheses	Basic	O(n)	O(n)	Stack matching
Min Stack	Basic	O(1)	O(n)	Min tracking
Infix-Postfix	Conversion	O(n)	O(n)	Precedence stack
Prefix-Infix	Conversion	O(n)	O(n)	Right-left traversal
Next Greater	Monotonic	O(n)	O(n)	Decreasing stack
Next Greater II	Monotonic	O(n)	O(n)	Circular array
Next Smaller	Monotonic	O(n)	O(n)	Stack with >=
Trapping Rainwater	Monotonic	O(n)	O(1)	Two pointers
Sum Subarray Min	Monotonic	O(n)	O(n)	Contribution
Asteroid Collision	Monotonic	O(n)	O(n)	Stack with collisions
Sum Subarray Ranges	Monotonic	O(n)	O(n)	Max - Min
Remove K Digits	Monotonic	O(n)	O(n)	Increasing stack
Largest Rectangle	Monotonic	O(n)	O(n)	Histogram
Maximal Rectangle	Monotonic	O(m*n)	O(n)	Histogram DP
Sliding Window Max	Implementation	O(n)	O(k)	Deque
Stock Span	Implementation	O(n)	O(n)	Previous greater
Celebrity Problem	Implementation	$O(n^2)$	O(1)	Elimination
LRU Cache	Implementation	O(1)	O(capacity)	HashMap + DLL
LFU Cache	Implementation	O(1)	O(capacity)	HashMap + Freq

Sliding Window & Two Pointer Summary

Problem	Difficulty	Time	Space	Key Technique
Longest Unique Substring	Medium	O(n)	O(1)	Map for last occurrence
Max Consecutive Ones III	Medium	O(n)	O(1)	Track zeros count
Fruit Into Baskets	Medium	O(n)	O(1)	Map for distinct count
Character Replacement	Medium	O(n)	O(1)	Frequency + maxFreq
Binary Subarray Sum	Medium	O(n)	O(n)	Prefix sum
Nice Subarrays	Medium	O(n)	O(1)	Convert to binary
All Three Characters	Medium	O(n)	O(1)	Frequency array
Max Points from Cards	Medium	O(n)	O(1)	Circular sliding window
Longest K Distinct	Hard	O(n)	O(k)	Map + shrink
K Different Integers	Hard	O(n)	O(k)	atMost(k) - atMost(k-1)
Minimum Window Substring	Hard	O(n)	O(m)	Map + count tracking
Minimum Window Subsequence	Hard	$O(n^2)$	O(1)	Subsequence matching

— End of enhanced Chapter 8: Stack & Queue / Sliding Window — remaining chapters continue in the full edition —